

CS222

Introduction to Multithread Programming

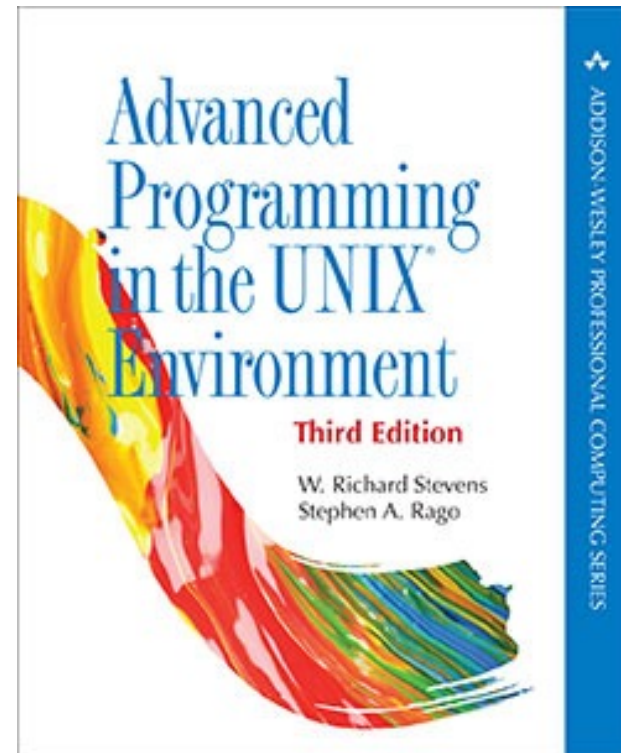
2nd Semester 2026

Kasidit Chanchio

Department of Computer Science

Textbook

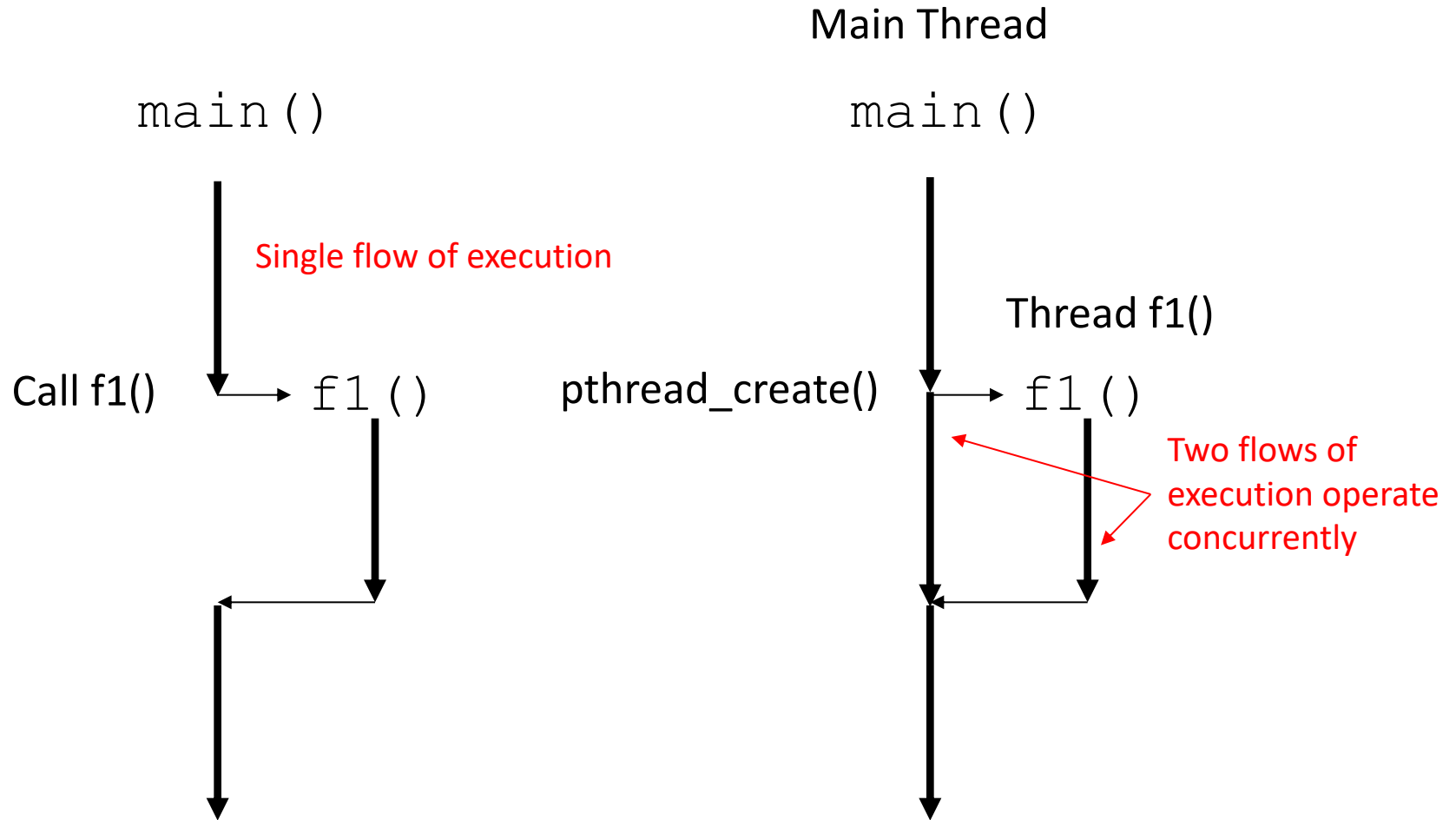
- Advanced Programming in the UNIX Environments
- <http://www.apuebook.com/>



Thread

- Thread คือการอนุญาตให้สร้าง *multiple flow of control* ใน process
- เรามักจะสร้างเธรดขึ้นมาเพื่อรองรับเหตุการณ์หลายเหตุการณ์ไปพร้อมๆกัน
- ในขณะที่โปรเซสจะแยกหน่วยความจำกัน เธรดเป็นสิ่งที่อยู่ในโปรเซสเดียวกัน มันจะแชร์หน่วยความจำของโปรเซสร่วมกัน และแชร์ไฟล์ที่โปรเซสเปิดด้วย
- เราสามารถสร้างเธรดหลายเธรดให้ประมวลผลบนหลาย CPU cores ไปพร้อมกันเพื่อให้โปรเซสผลิตผลงานได้มากขึ้น
- นอกจากนั้น เรายังสามารถสร้างเธรดหลายเธรด และกำหนดให้แต่ละเธรดทำหน้าที่ต่างกันเช่น เธรดหนึ่งติดต่อผู้ใช้ อีกเธรดหนึ่งประมวลผล อีกเธรดหนึ่งอ่านเขียนข้อมูลกับอุปกรณ์ I/O เป็นต้น

Process and Threads



Types of Thread

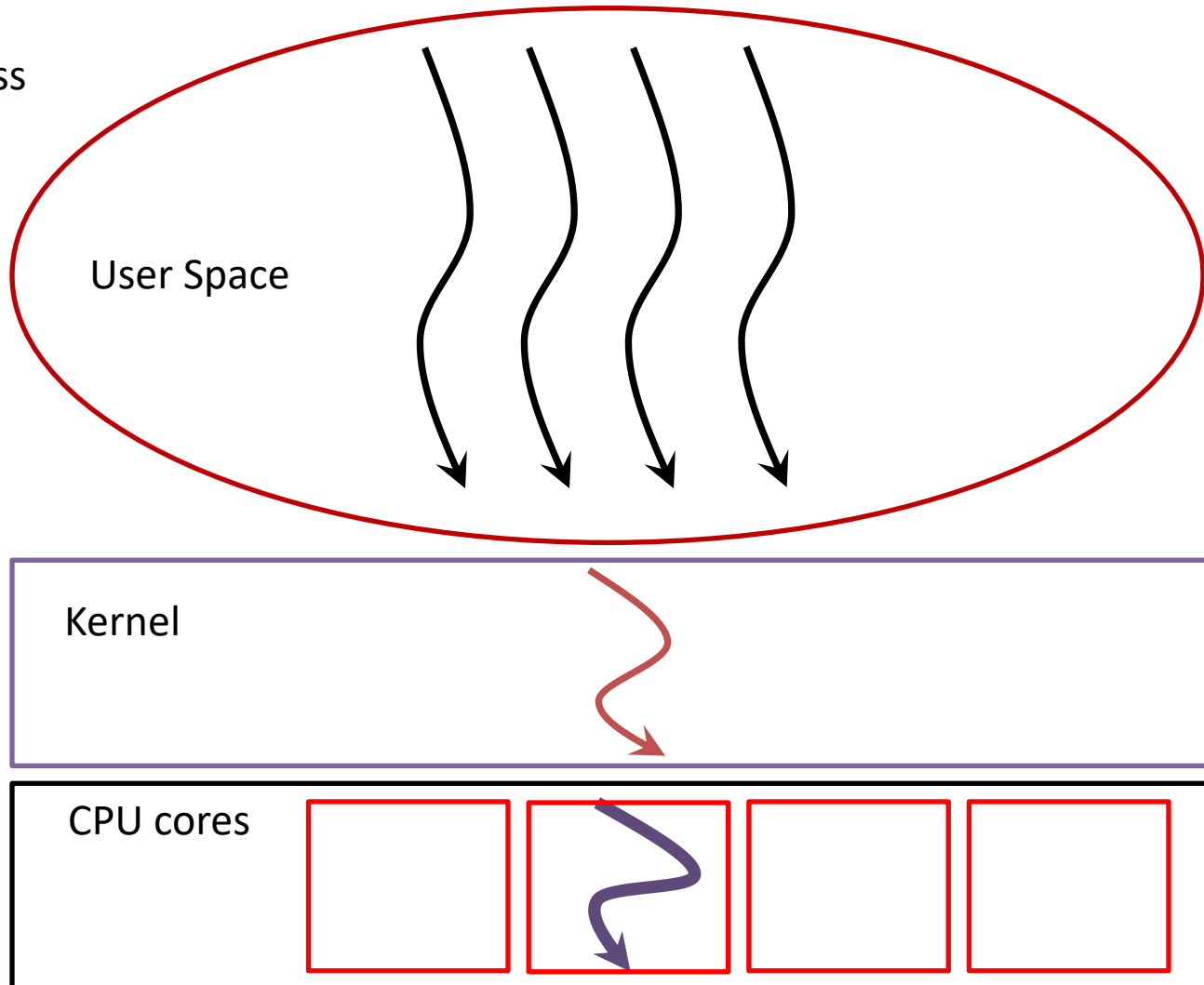
- There are 3 types:
 - *User-Level Thread*,
 - *Kernel-Level Thread*, and
 - *Light-weighted Process (Many to Many Model)*
- Remind that process is preemptive in multiprogramming OS.
- ***Process context switching is costly*** due to the isolation principle where each process has separate memory space.
- Process's context switching costs include:
 - Save/restore register contents
 - Changing page table, caching, virtual memory
 - TLB flushing

1. User-Level Thread

- *Thread scheduling is implemented in process user space.*
Threading is emulated in process.
- Thus, context switching is low:
 - *Context switching is performed in user space:* no need to change CPU execution mode to access kernel's memory
- Disadvantages:
 - A blocking I/O suspends the entire process, even though another thread is ready to run
 - Cannot exploit multi-processing capability of HW

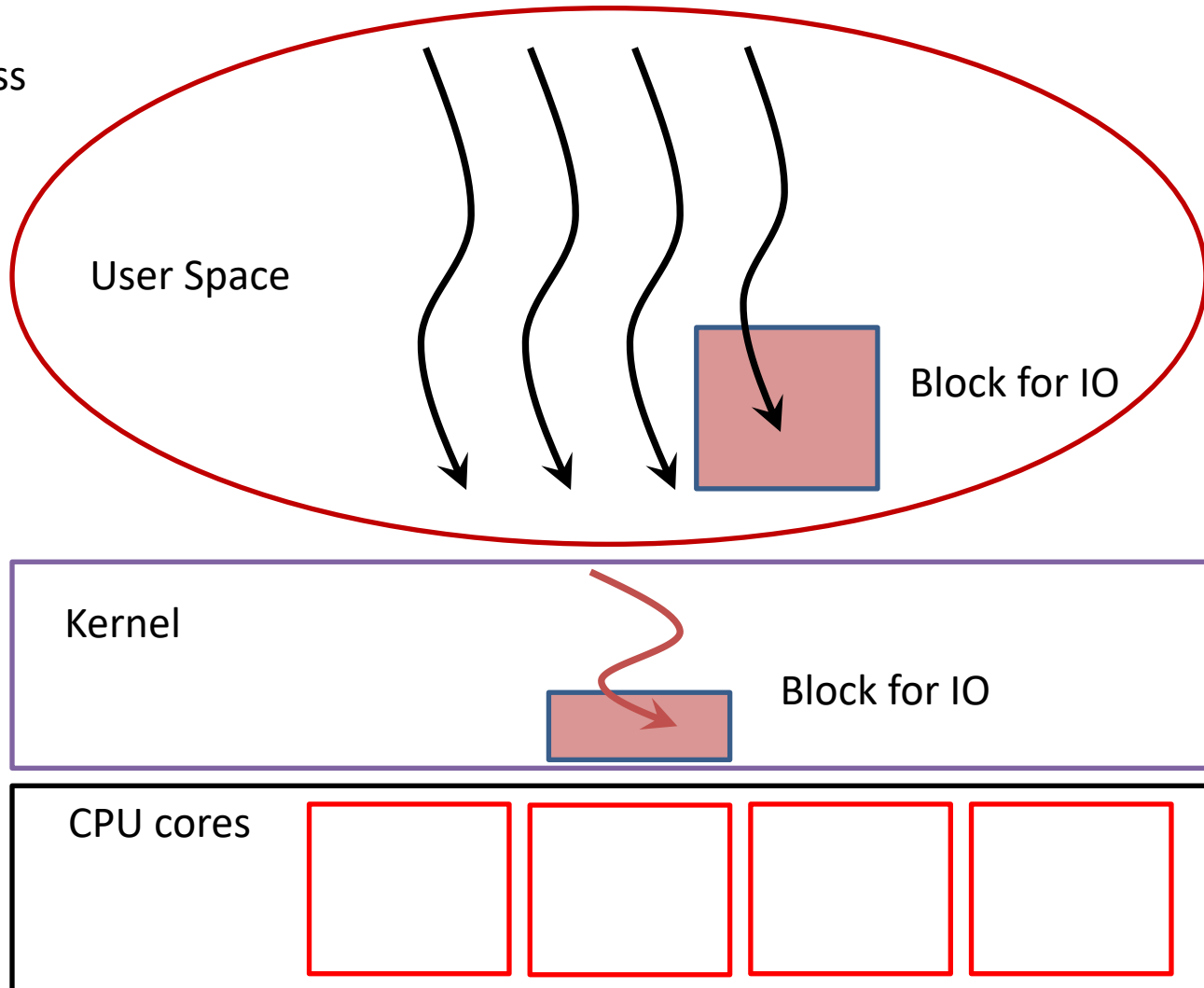
User-level Thread

A Process



User-level Thread

A Process

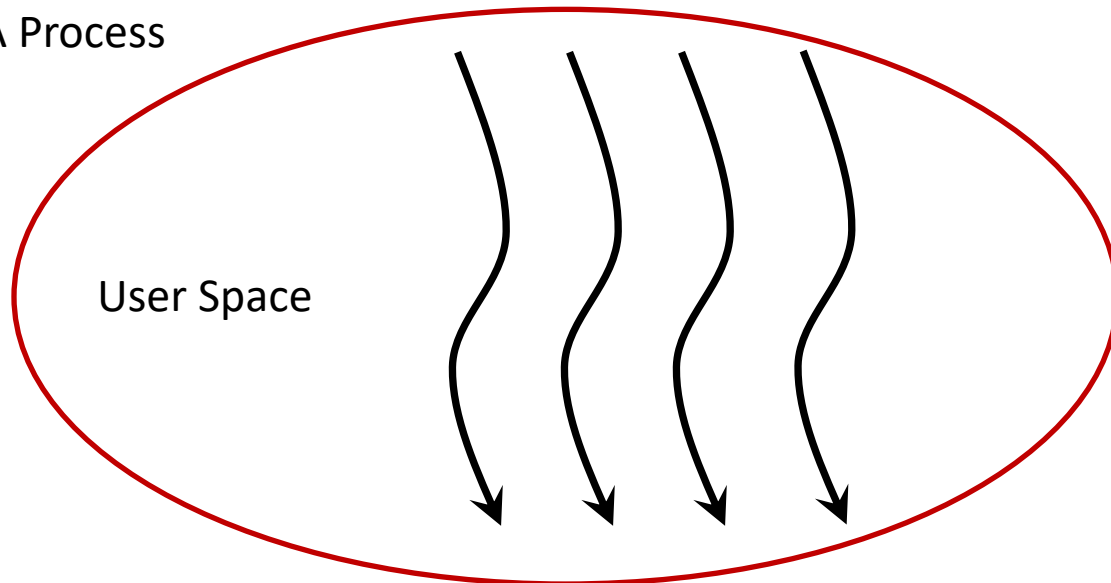


2. Kernel-Level Thread

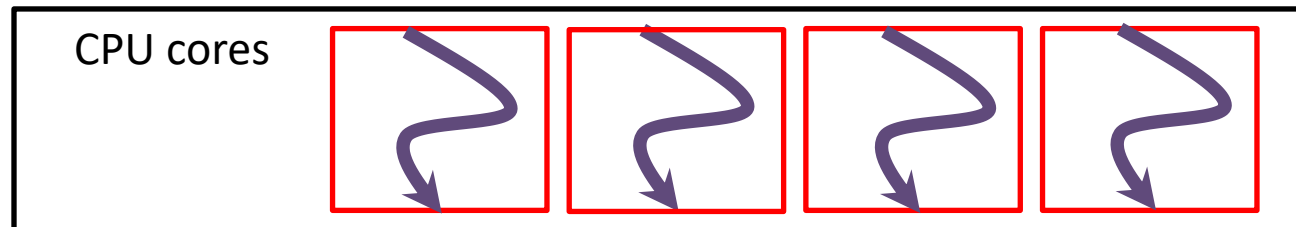
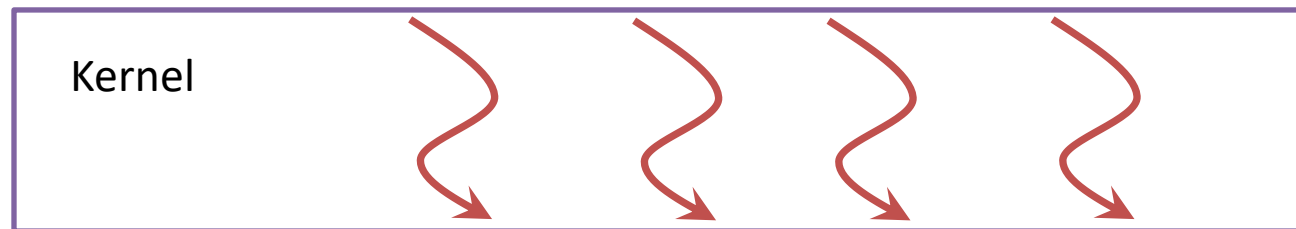
- *Thread scheduling is implemented in kernel.*
- Linux ใช้ Kernel-Level Thread
- Disadvantages:
 - *context switching is cost is high* but not as much as process's context switching.
- Advantages:
 - *When a thread is blocked for I/O, another thread can run*
 - *Multiple threads on the same process can run concurrently on different CPU cores or processors*

Kernel-level Thread

A Process

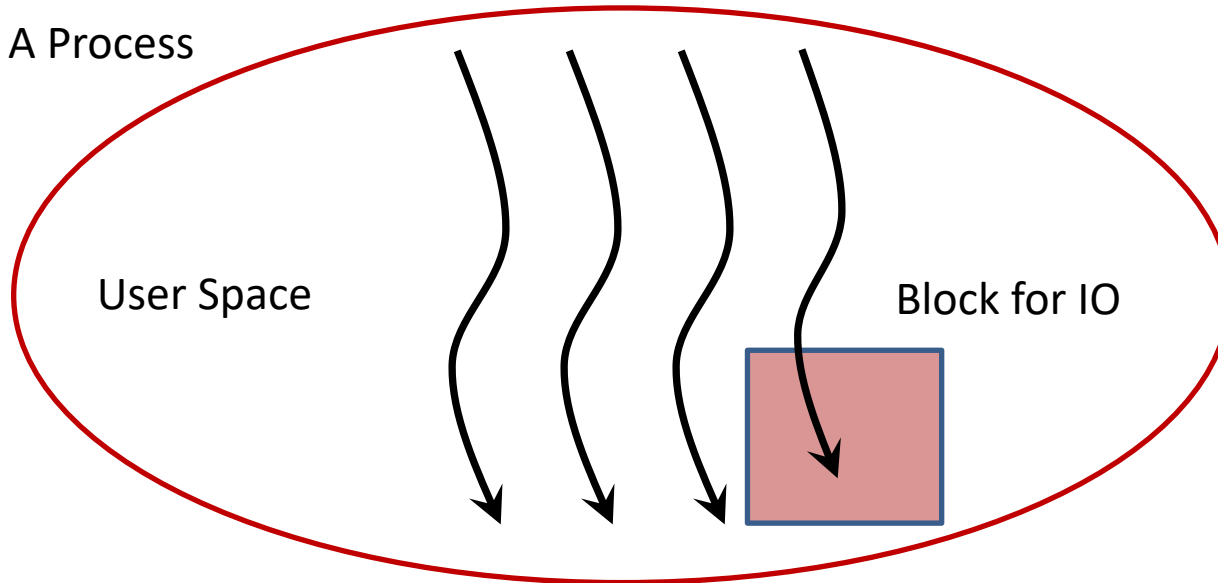


Multiple threads on the same process can run concurrently on different CPU cores or processors

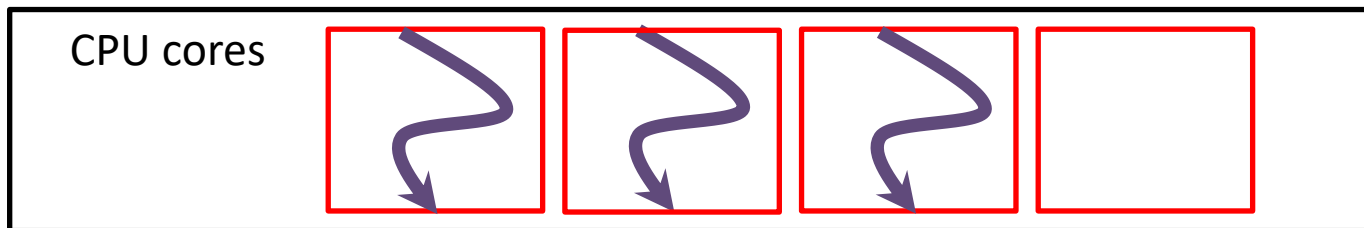
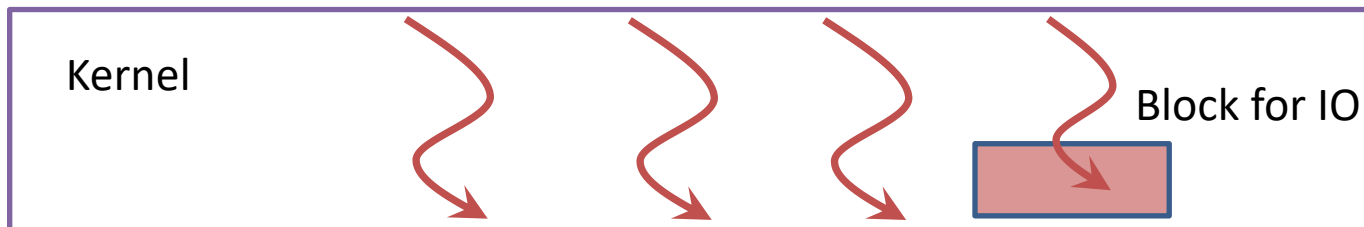


Kernel-level Thread

A Process

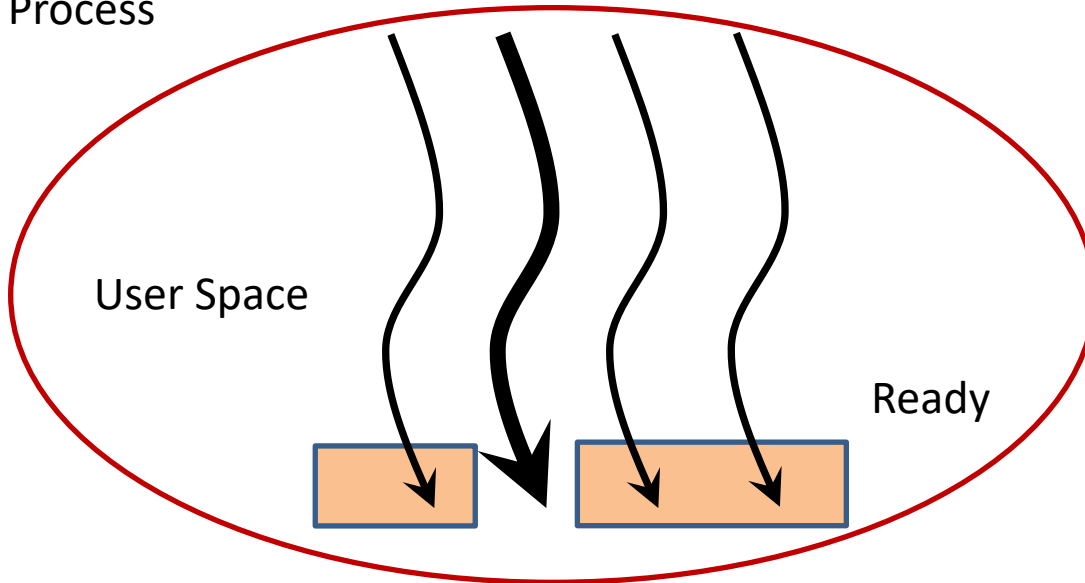


When a thread is blocked for I/O, another thread can run

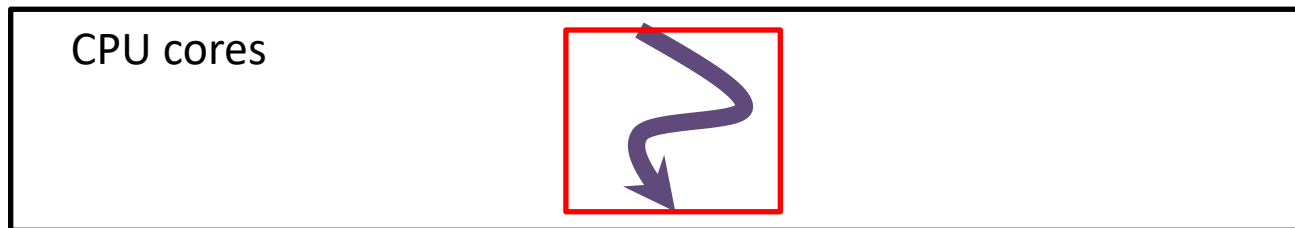
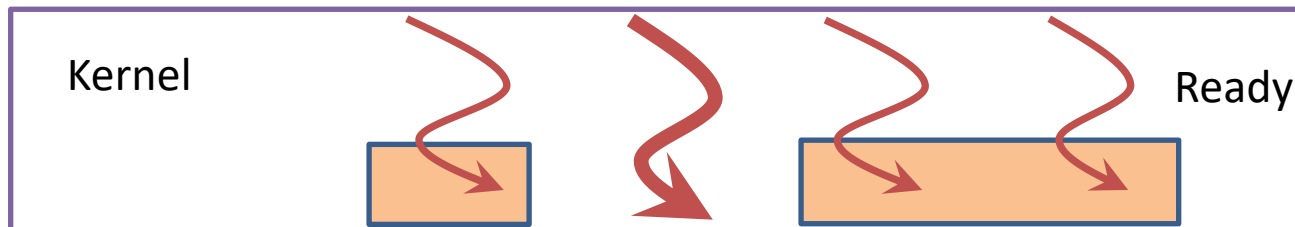


Kernel-level Thread

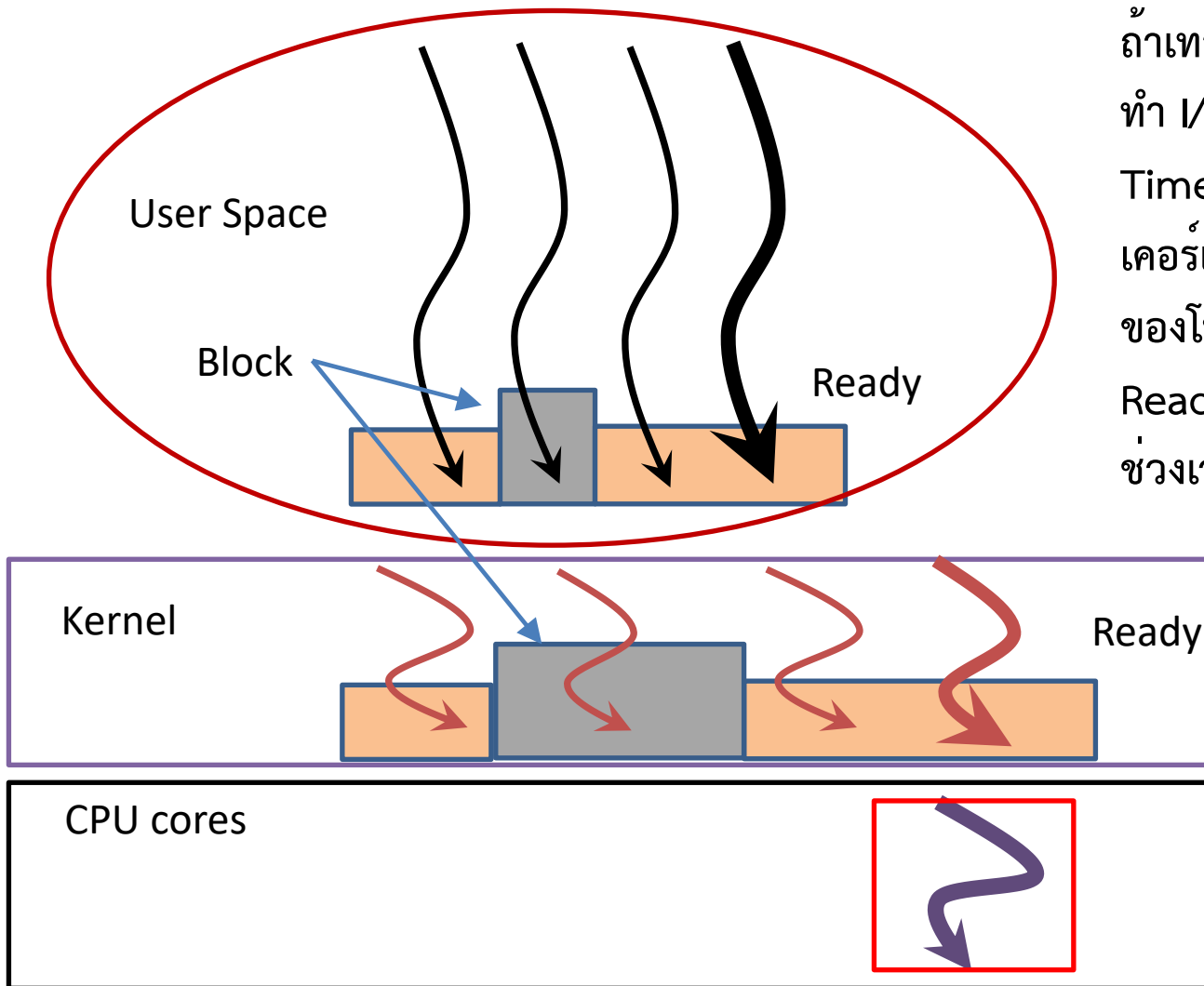
A Process



สมมติว่า เทรดหนึ่ง ของโปรเซส
ได้ Time Slice T และได้ใช้ซีพียู
คอร์หนึ่งในขณะที่เทรดอื่นของโพร
เซสเดียวกันรอใน ready
queue



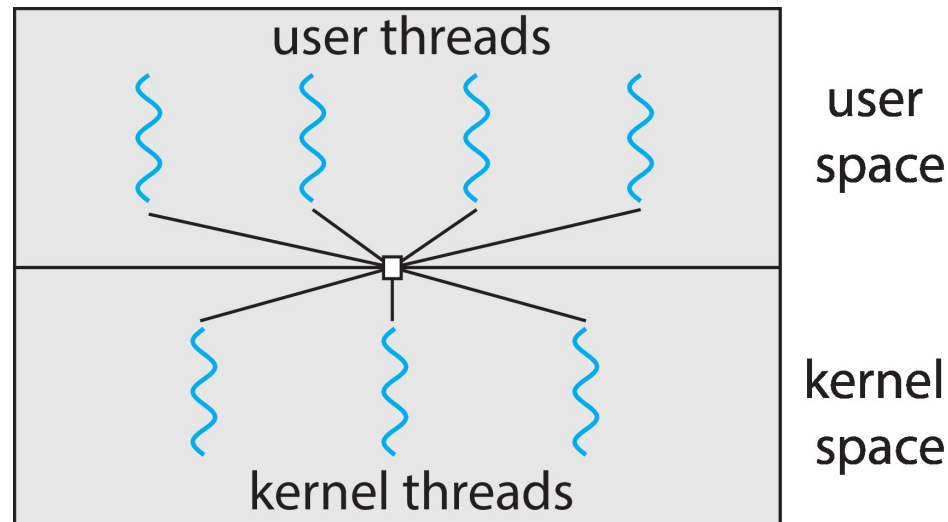
Kernel-level Thread



ถ้าเธรดนั้น block เนื่องจากต้อง
ทำ I/O operations และเวลา
Time Slice ของมันยังเหลืออยู่
เคอร์เนลสามารถเลือกเธรดอื่น
ของโปรเซสเดียวกันที่รอใน
Ready queue เข้ามารันต่อใน
ช่วงเวลา Time Slice ที่เหลือ

3. Many-to-Many Model

- Allows many user level threads to be mapped to many kernel threads
- โพรเซสจะสร้าง Kernel-level threads จำนวนหนึ่งเช่น 3 เทรด ในภาพ แล้วสร้าง User-level threads (ULT) จำนวนหนึ่ง เช่น 100 เทรด แล้ว ใช้กลไก Scheduling ภายในโพรเซสสั่งให้ Kernel-level threads เลือกทีละ 3 ULT มารัน
- Scheduler อาจสั่งให้คอร์เนลเทรดที่กำลังรัน UTL หนึ่งอยู่เปลี่ยนไปรัน ULT อื่นได้ เมื่อ ULT เดิมรันคำสั่ง yield หรือรันถึงตำแหน่งในโค้ดที่คอร์เนลเข้ามารันได้

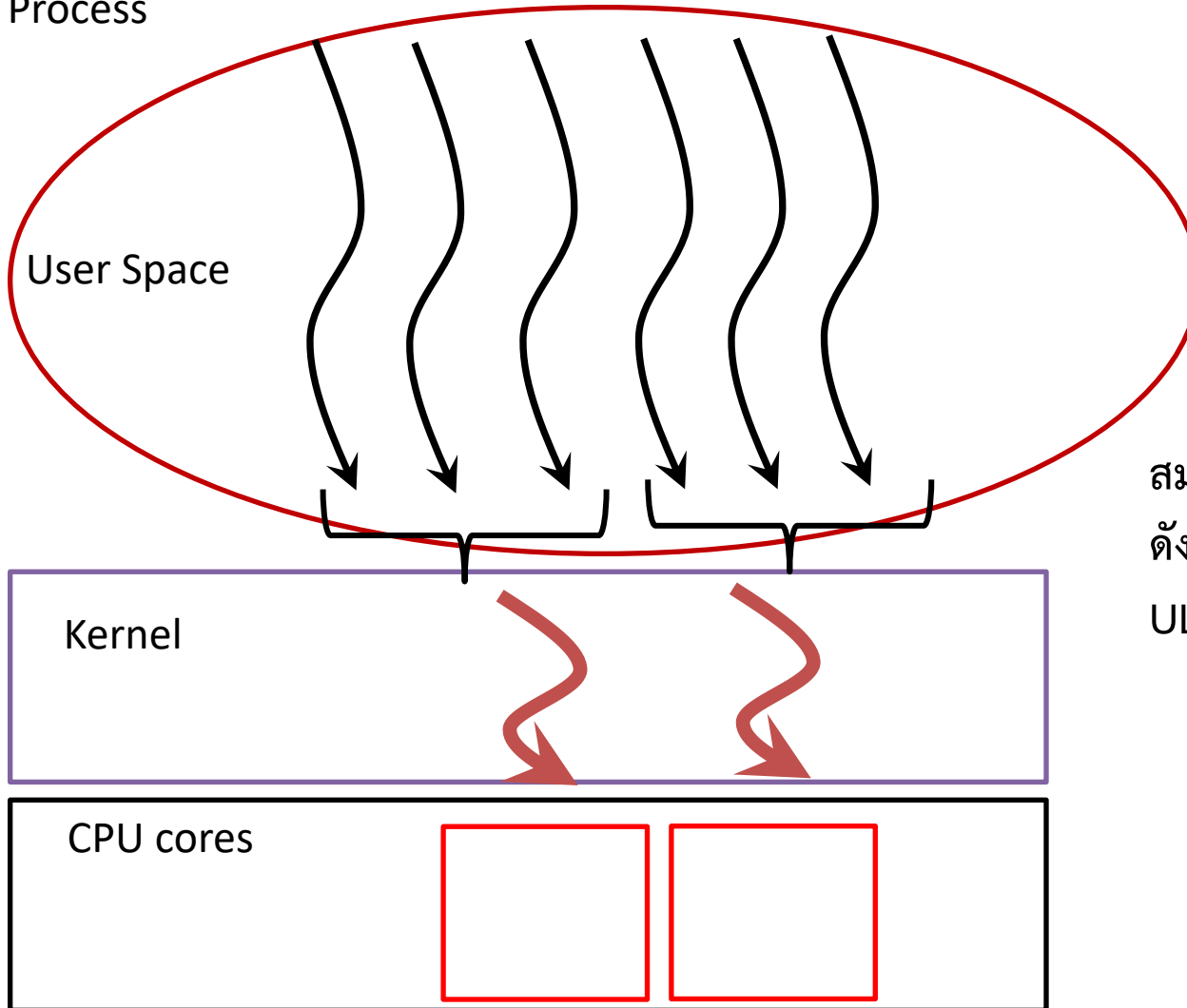


Many-to-Many Model

- ยกตัวอย่างเช่น Lightweighted Process บน Solaris และ *ThreadFiber* package บน Windows ซึ่งทั้งคู่ไม่ประสบความสำเร็จ
 - ปัญหาหนึ่งคือ Kernel-level thread (KLT) อาจ block รอ I/O แล้วทุก user-thread ที่ถูกกำหนดให้ใช้ kernel-level thread นั้นก็ต้องรอด้วย
 - การ Implement เป็น Programming Library ทำให้ใช้งานยาก
- ประสบความสำเร็จใน Goroutine ใน Golang เนื่องจาก
 - ใช้ Goroutine Machine Processor (GMP) Model สามารถแก้ปัญหา block รอ I/O ได้
 - Scheduler ถูก Implement ใน Go runtime ซึ่งเป็นส่วนหนึ่งของ Language
 - Goroutine ใช้ Stack segment 2KB ในขณะที่ KLT ใช้ 1 MB Stack segment

Many-to-Many Model

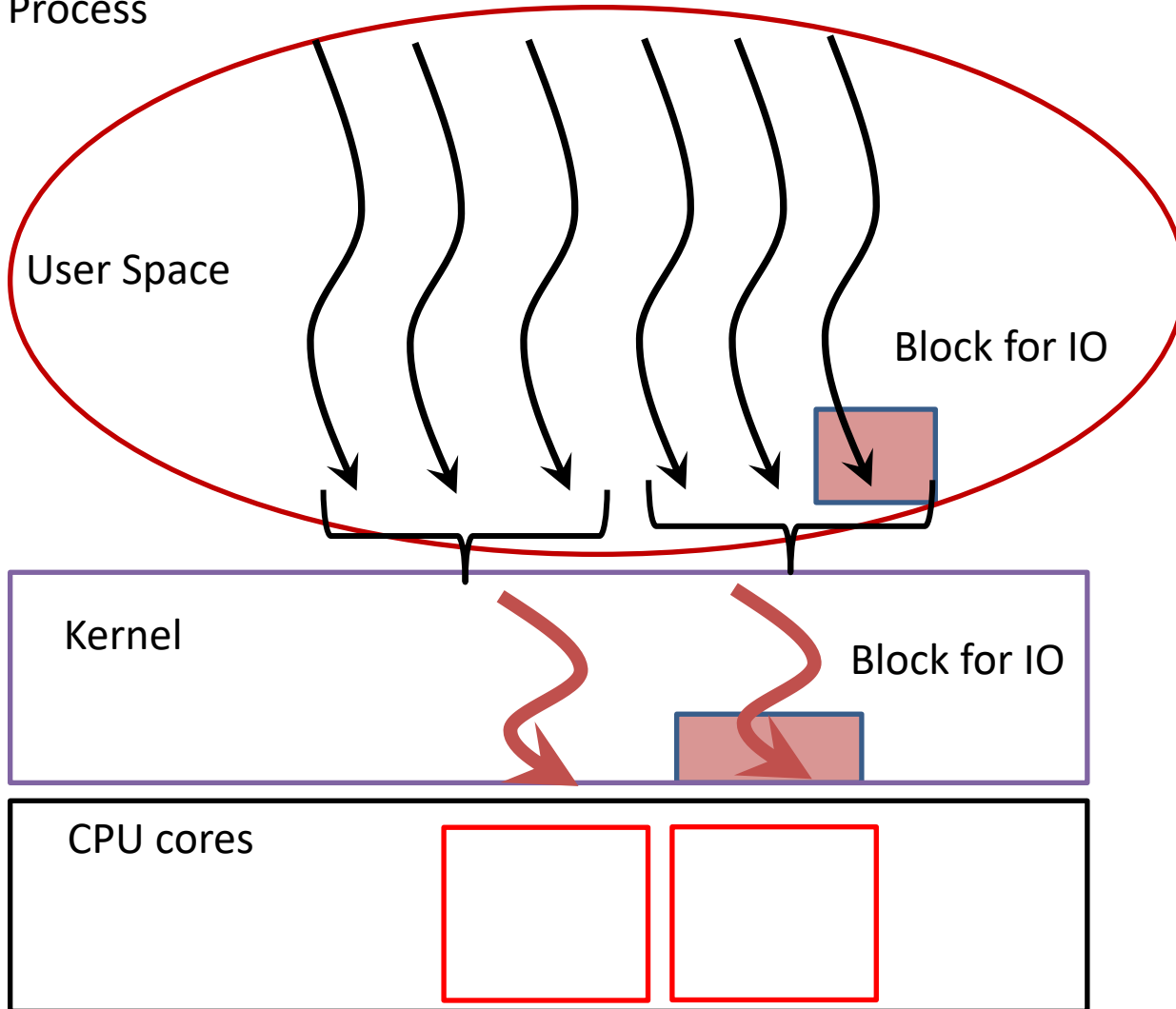
A Process



สมมติว่า 6 ULT ใช้ 2 KLT
ดังนั้น scheduler จะ assign 3
ULT ให้แต่ละ KLT

Many-to-Many Model

A Process

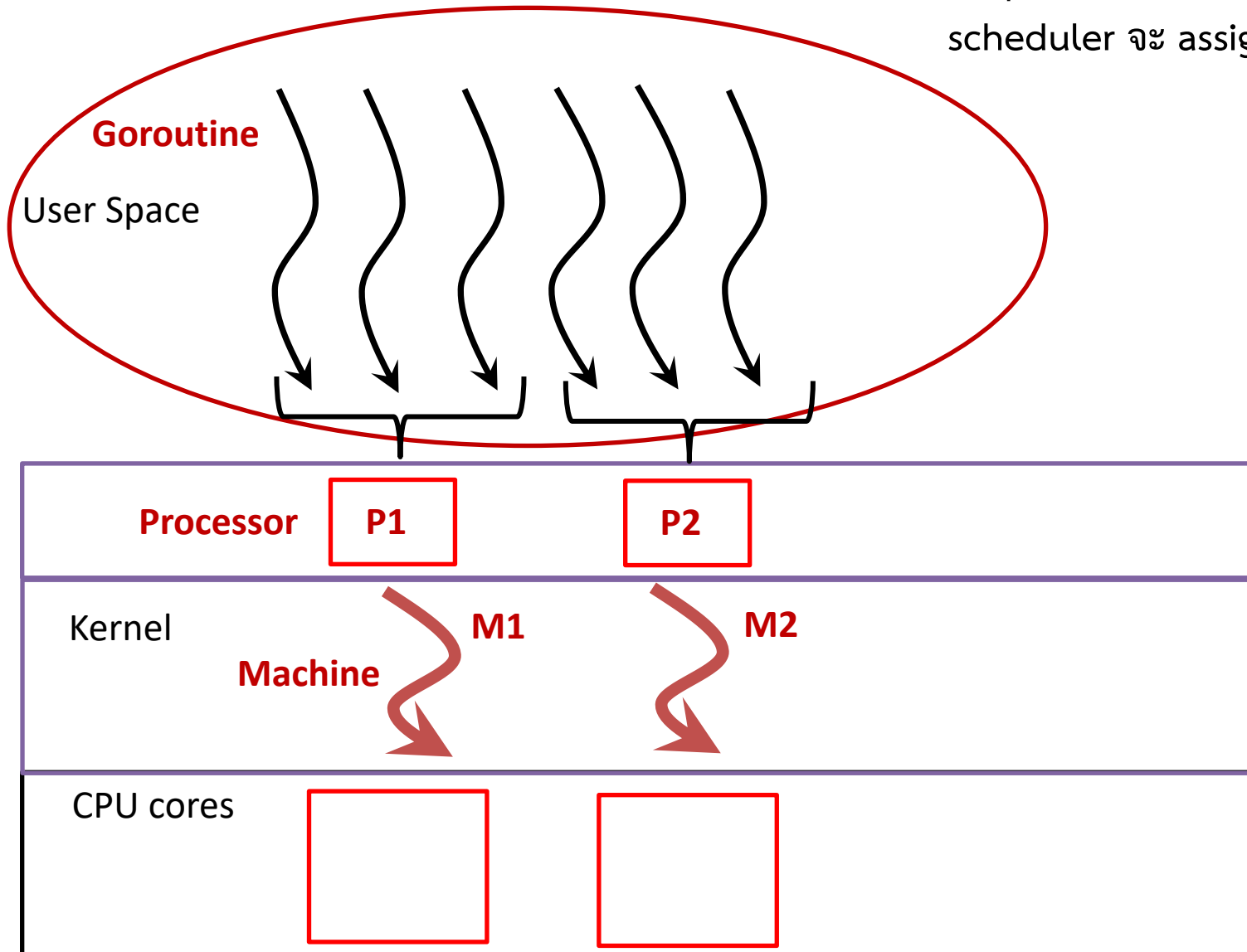


ถ้ามี 1 KLT block รอ I/O
หรือเหตุการณ์อื่นเช่น mutex
ทั้ง 3 ULT ที่ถูก assign ให้มัน
ก็ต้องรอไปด้วย

Goroutine

A Process

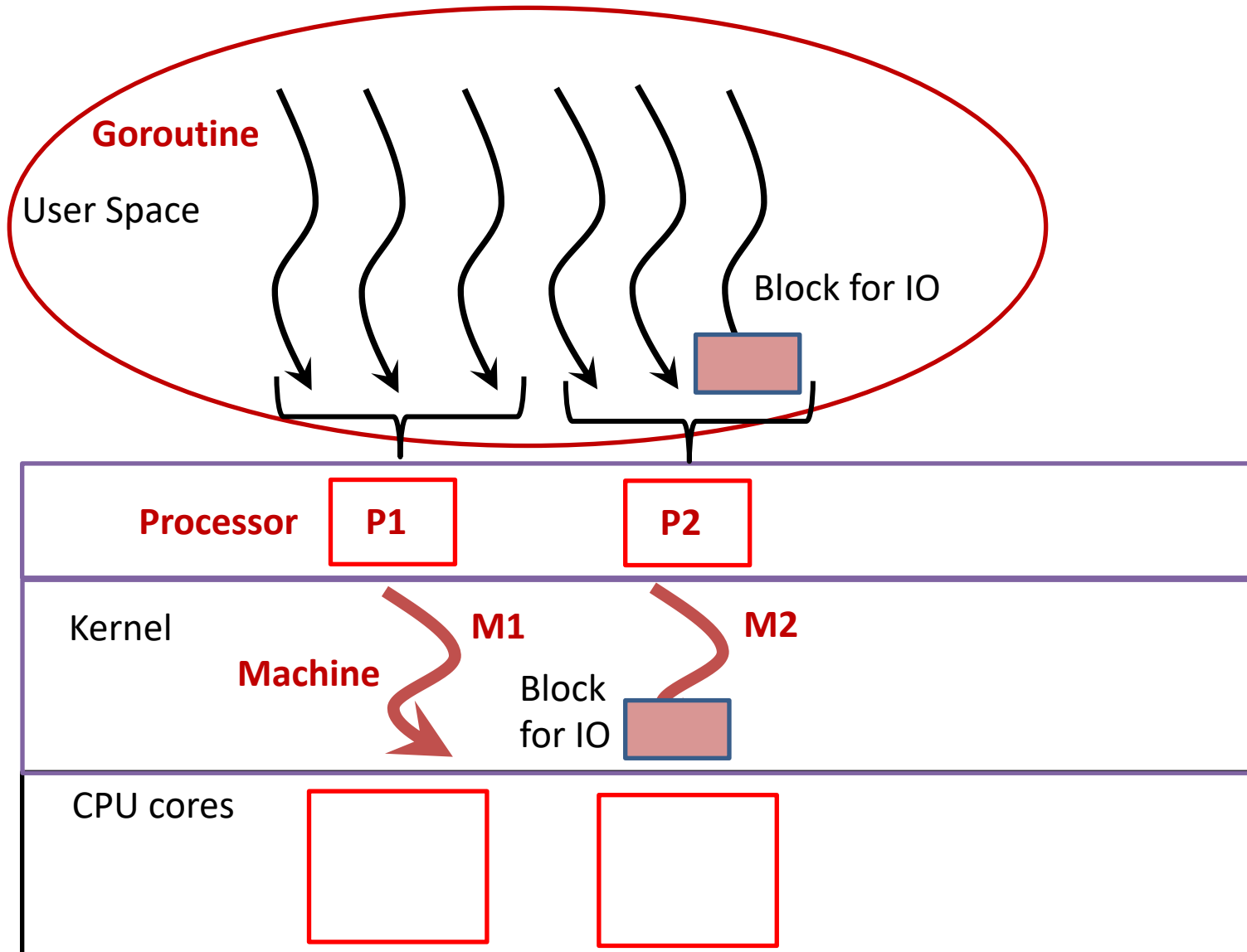
สมมุติว่า 6 G ใช้ 2 P ดังนั้น
scheduler จะ assign 3 G ให้แต่ละ P



Goroutine

A Process

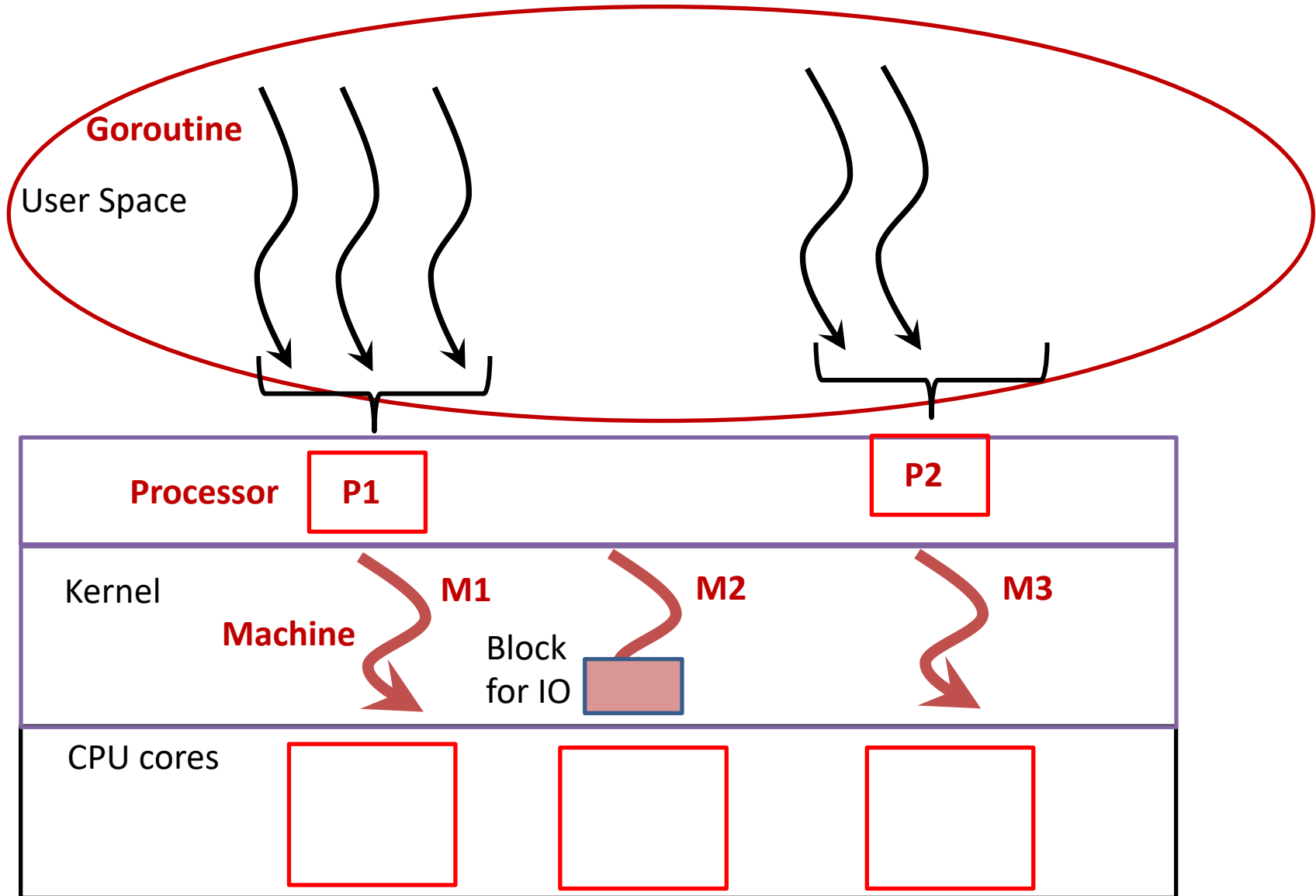
ถ้า M2 block ...



Goroutine

A Process

ถ้า M2 blocks, go runtime จะ hand off P2 ไปให้ new M หรือ M3 ในภาพ



Linux Threads

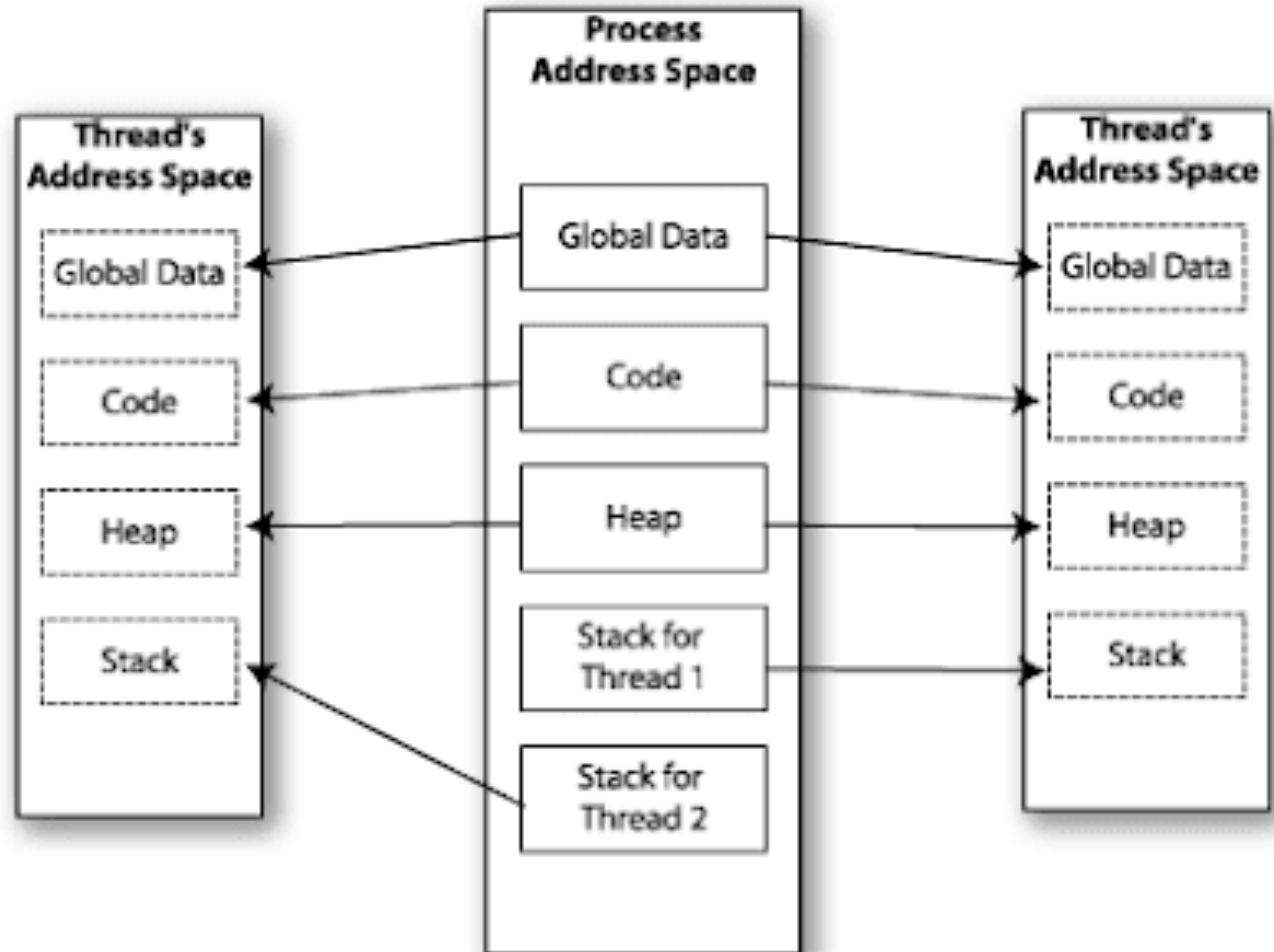
- Linux refers to them as *tasks* rather than *threads*
- Thread creation is done through `clone()` system call
- `clone()` allows a child task to share the address space of the parent task (process)
 - Flags control behavior

flag	meaning
CLONE_FS	File-system information is shared.
CLONE_VM	The same memory space is shared.
CLONE_SIGHAND	Signal handlers are shared.
CLONE_FILES	The set of open files is shared.

- `struct task_struct` points to process data structures (shared or unique)

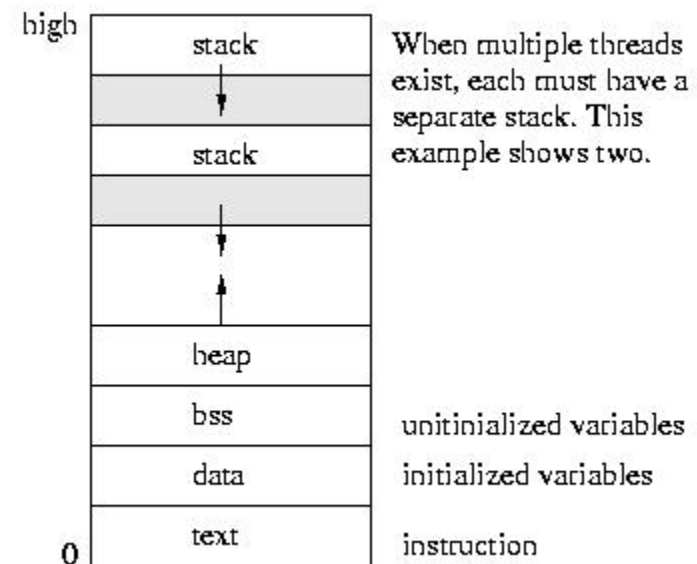
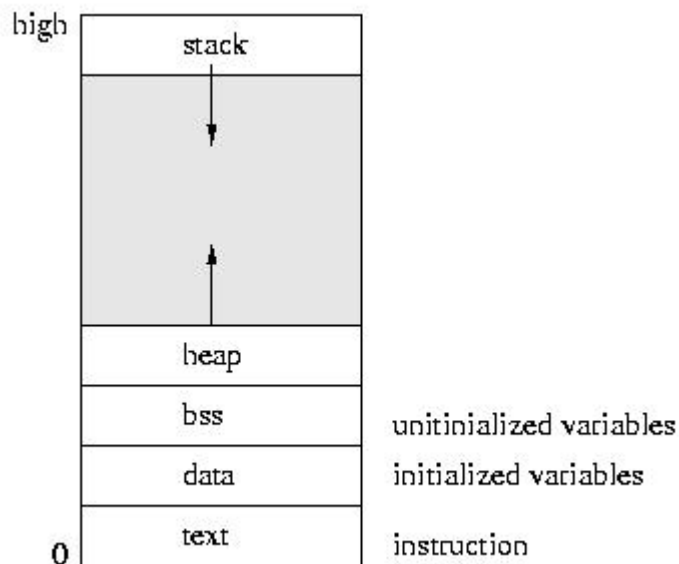
Memory Layout

- Share global, code, heap
- Divide stack



Process Memory Layout

- Threads share global data, code, and heap segments of a process.
- Each thread has a separate stack segment.



Reference: <http://www.andrew.cmu.edu/course/15-310/applications/ln/lecture17.html>

Global variables

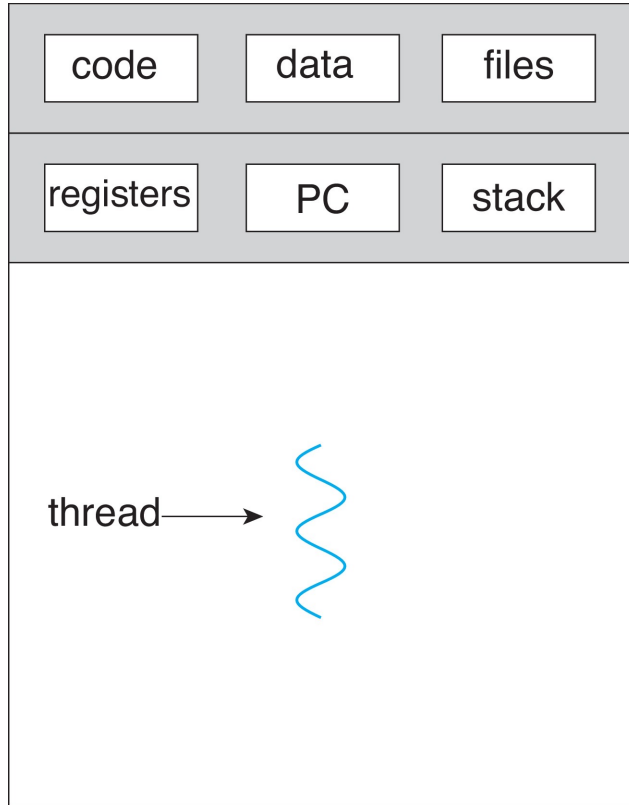
- Can introduce subtle and confusing bugs!
- Limit use of global variables to situations in which they're really needed.
 - Shared variables.



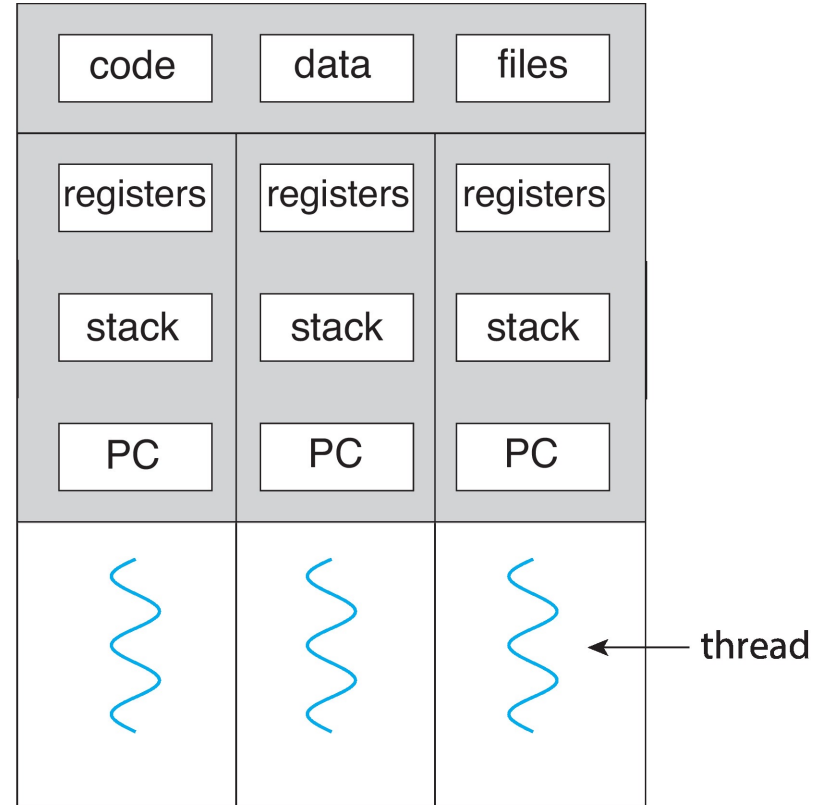
Motivation

- Most modern applications are multithreaded
- **Threads run within application**
- Multiple tasks with the application can be implemented by separate threads
 - Update display
 - Fetch data
 - Spell checking
 - Answer a network request
- Process creation is heavy-weight while thread creation is light-weight
- Can simplify code, increase efficiency
- Kernels are generally multithreaded
 - ใน OS เช่น FreeBSD หลังจากโปรเซส 0 สร้างโปรเซส 1 (init) มันจะสร้าง threads ขึ้นจำนวนหนึ่งเพื่อดูแลการใช้งานระบบคอมพิวเตอร์

Single and Multithreaded Processes

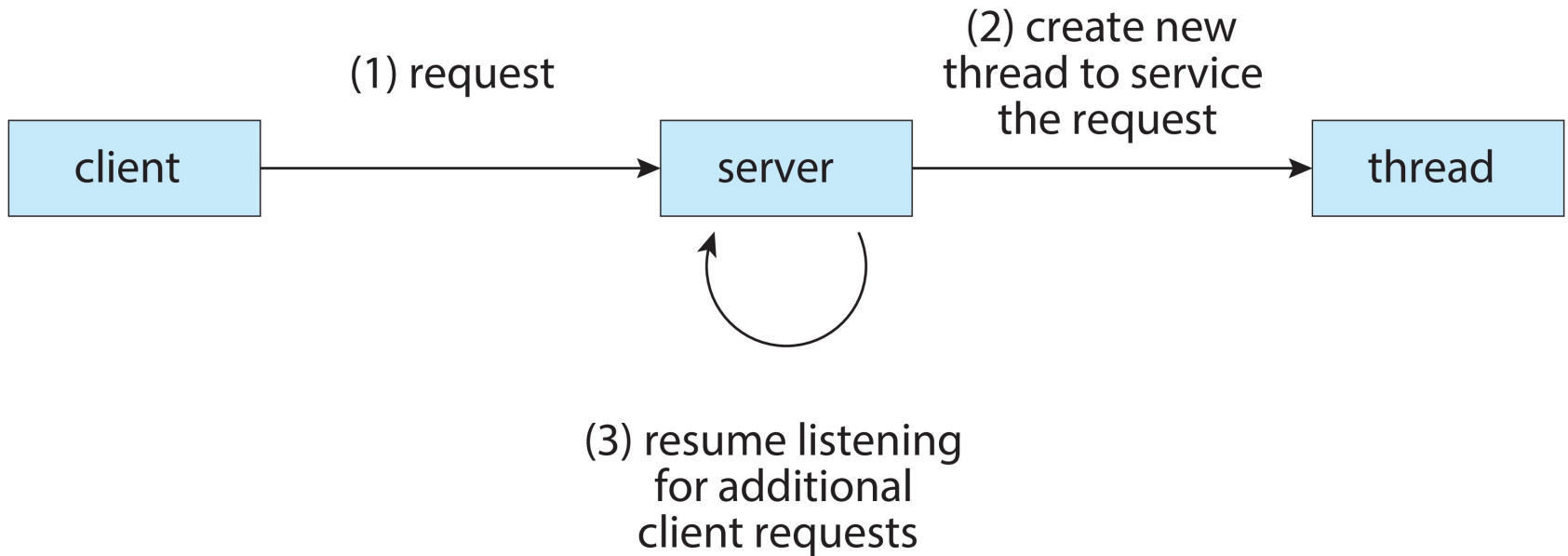


single-threaded process



multithreaded process

Multithreaded Server Architecture



Benefits

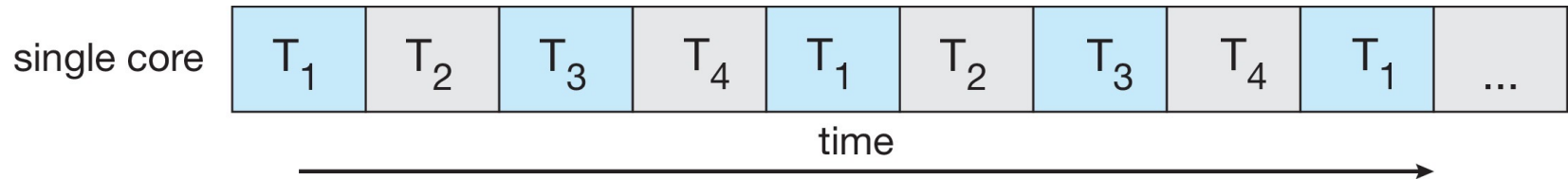
- **Responsiveness** - may allow continued execution if part of process is blocked, especially important for user interfaces
- **Resource Sharing** - threads share resources of process, easier than shared memory or message passing
- **Economy** - cheaper than process creation, thread switching lower overhead than context switching
- **Scalability** - process can take advantage of multicore architectures

Multicore Programming

- **Multicore** or **multiprocessor** systems putting pressure on programmers, challenges include:
 - Dividing activities
 - Balance
 - Data splitting
 - Data dependency
 - Testing and debugging
- **Parallelism** implies a system can perform more than one task simultaneously
- **Concurrency** supports more than one task making progress
 - Single processor / core, scheduler providing concurrency

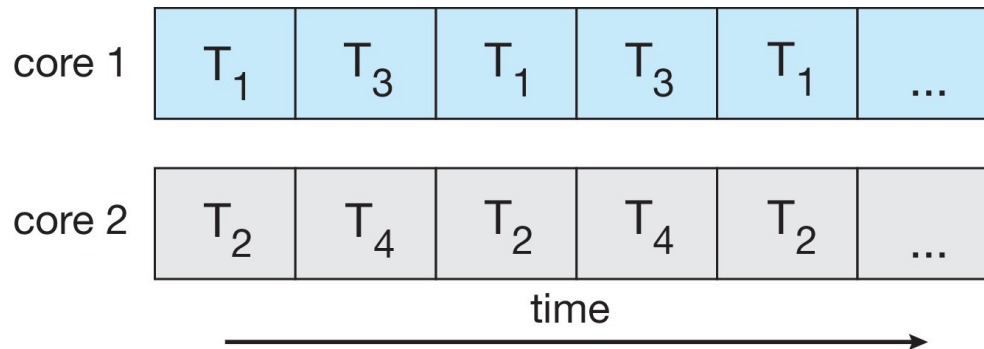
Concurrency vs. Parallelism

- **Concurrent execution on single-core system:**



- ทุกโปรเซสมีความคืบหน้า

- **Parallelism on a multi-core system:**

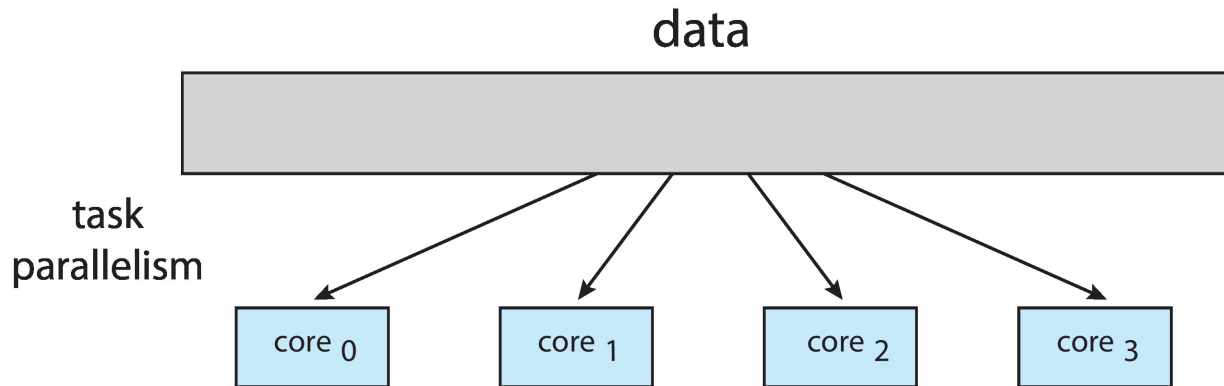
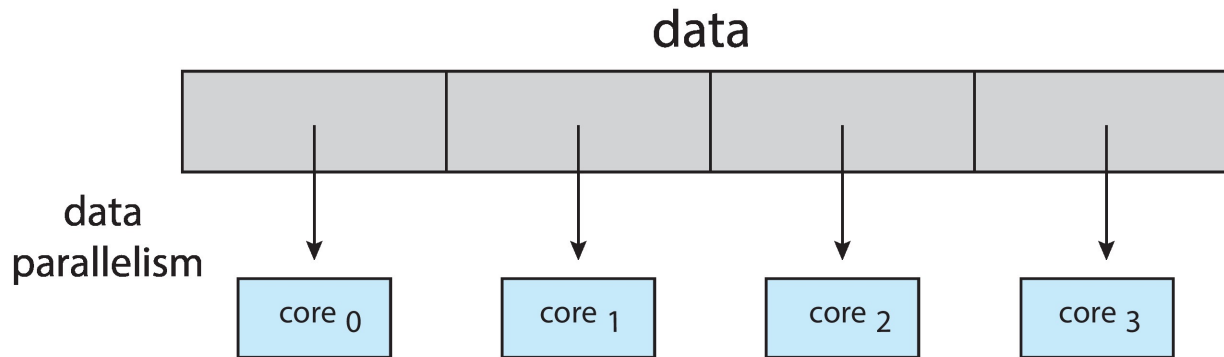


- หลายโปรเซสทำงานไปพร้อมกันจริงๆ

Multicore Programming

- Types of parallelism
 - Data parallelism – distributes subsets of the same data across multiple cores, same operation on each
 - Task parallelism – distributing threads across cores, each thread performing unique operation

Data and Task Parallelism



ธรรมชาติของการประมวลผลแบบขนาน (parallel)

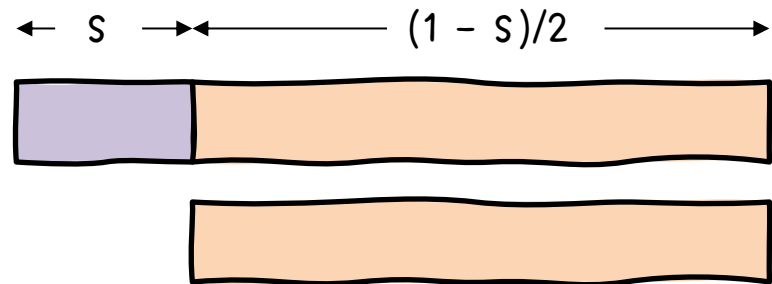
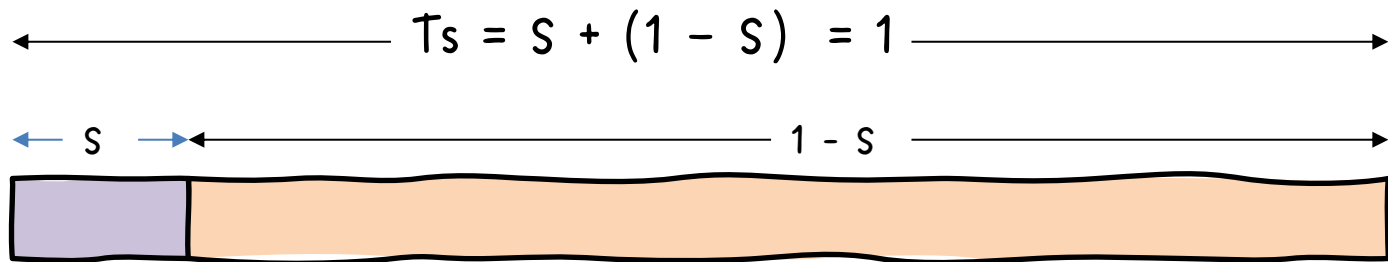
- สมมติว่า นศ มีบริษัทก่อสร้าง มีคนงาน 10 คน
- บริษัทสร้างบ้านหนึ่งหลัง ใช้เวลา 1 เดือน
- เปรียบเหมือน นศ มี CPU cores 10 cores (หรือ 10 processors)
- มีงาน W work และใช้เวลา T time จึงทำงานเสร็จ
- ถ้ามีคนงานเร่งอยากจะได้บ้านเร็วมาก แล้ว นศ เริ่มคำนวณ
- สร้างบ้าน 1 หลัง คนงาน 10 คน ใช้เวลา 720 ชม (30 วัน)
 - แสดงว่างานรวมถ้าทำคนเดียวต้องใช้เวลา $720 \times 10 = 7200$ ชม
- ดังนั้น ถ้าอยากให้บ้านเสร็จใน 1 ชม ก็ต้องใช้คน 7200 คน

ธรรมชาติของการประมวลผลแบบขนาน (parallel)

- ถ้าอยากให้บ้านเสร็จภายใน 1 นาที ต้องใช้ 7200x60 คน ????
- เหมือน นศ มีงานกลุ่มที่ทำ 3 คนเสร็จใน 1 อาทิตย์ ถ้าเพิ่มคน 1000 คนจะเสร็จเร็วขึ้นไหม
- จะเห็นว่าเป็นไปไม่ได้ เพราะ ในการทำงานมีงานสองประเภท
 - งานที่แบ่งกันไปได้
 - งานที่แบ่งกันไม่ได้ต้องทำงานคนเดียว
- ในการทำงานแบบขนานทำให้เกิดงานแบบใหม่ (ไม่มีในการทำงานคนเดียว)
 - การแบ่งงานและบริหารจัดการ
 - การรอกัน
 - การสื่อสารประสานงาน
 - การรวบรวมผลงาน

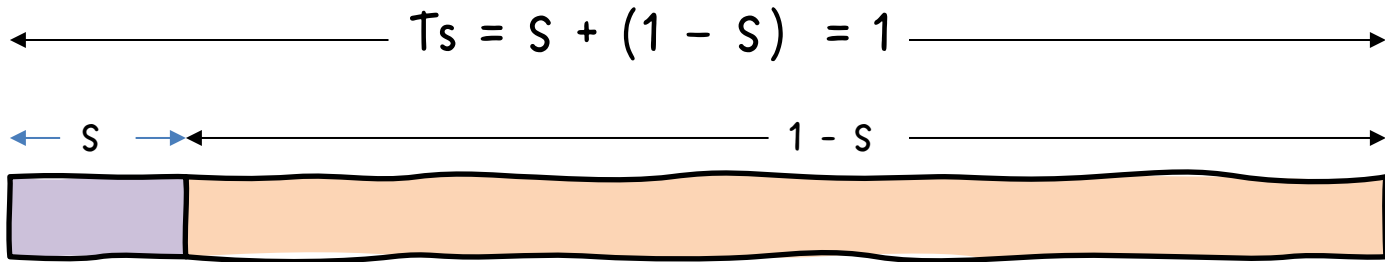
Amdahl's Law

- สมมุติว่าเวลาที่ใช้ประมวลผลแบบ sequential คือ 1
- สมมุติว่างานที่แบ่งไม่ได้คือ S
- $Work = 1$
- S is serial portion
- N processing cores



$$T_p = S + (1 - S)/N = 1$$

Amdahl's Law

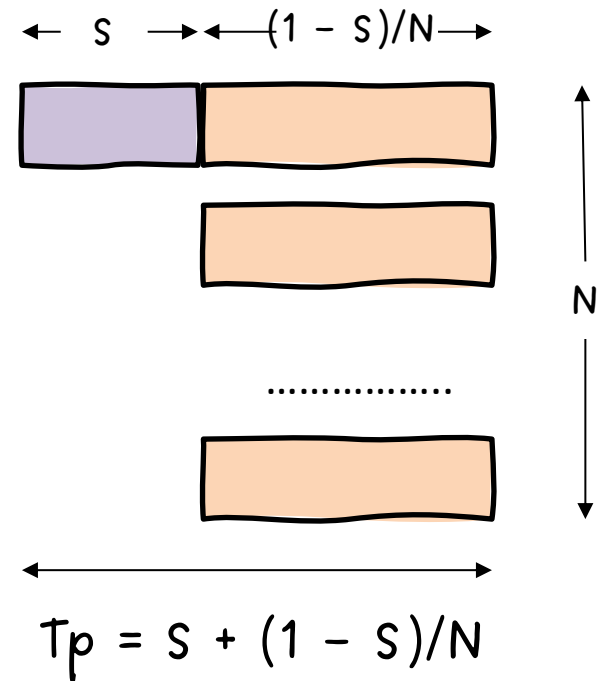


- Speedup $S(N)$ เมื่อใช้ N processors แล้วเร็วขึ้นกี่เท่า

- $S(N) = T_s / T_p$

$$= \frac{1}{S + \frac{(1-S)}{N}}$$

- เมื่อ N สูง $S(N) \rightarrow 1/S$



Amdahl's Law

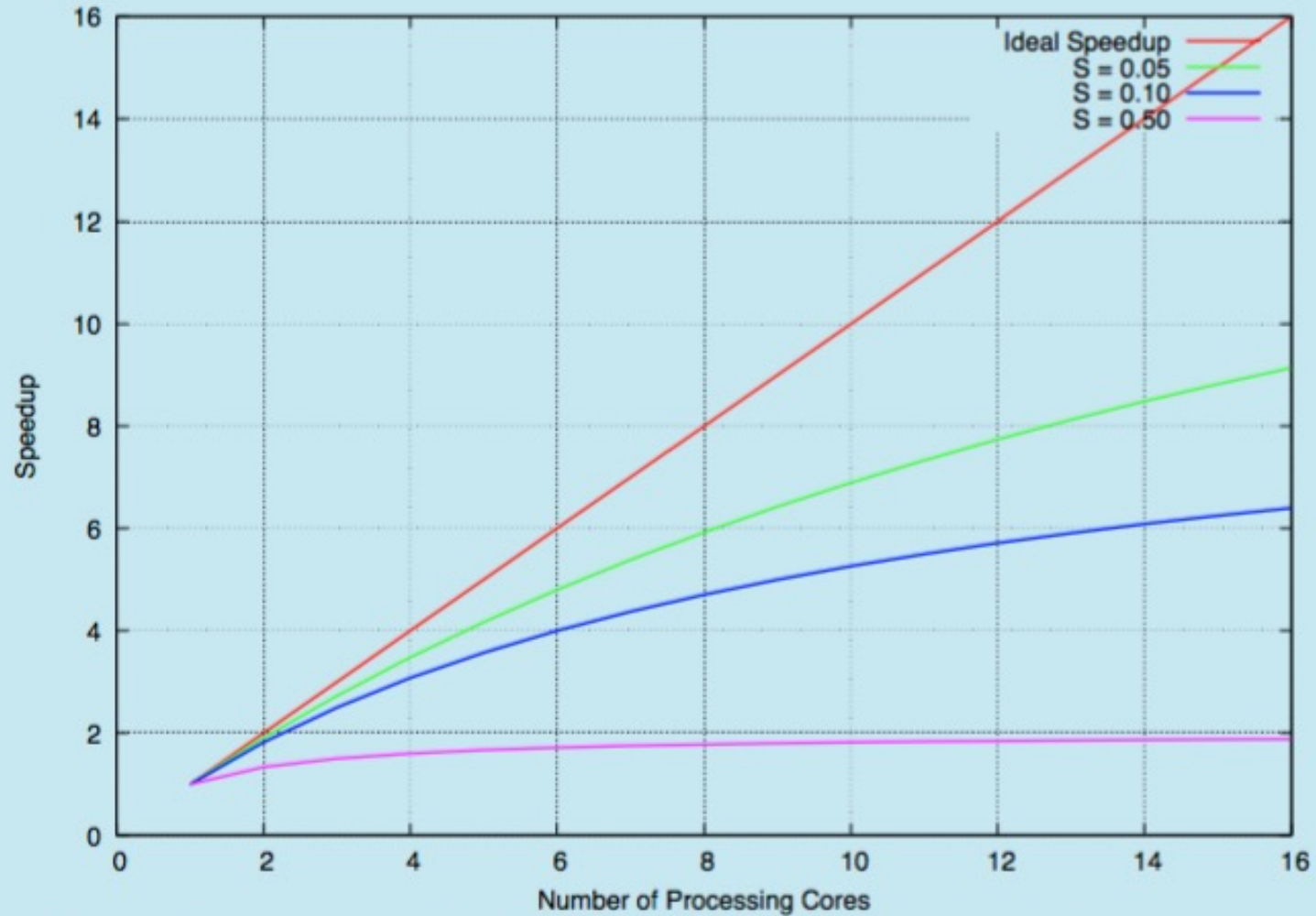
- Identifies performance gains from adding additional cores to an application that has both serial and parallel components
- S is serial portion
- N processing cores

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

- That is, if application is 75% parallel / 25% serial, moving from 1 to 2 cores results in speedup of 1.6 times
- As N approaches infinity, speedup approaches $1 / S$

Serial portion of an application has disproportionate effect on performance gained by adding additional cores

Amdahl's Law



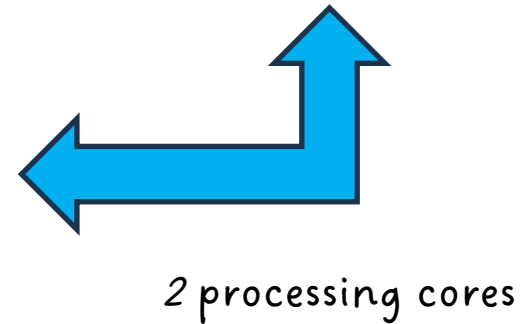
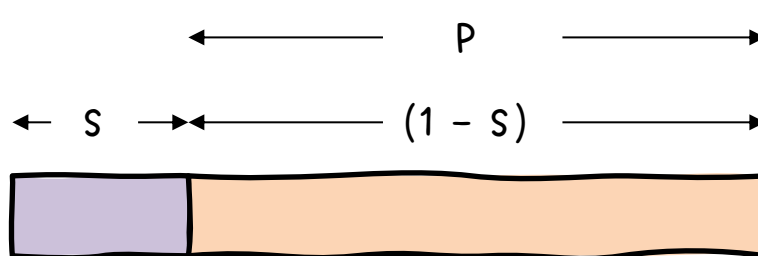
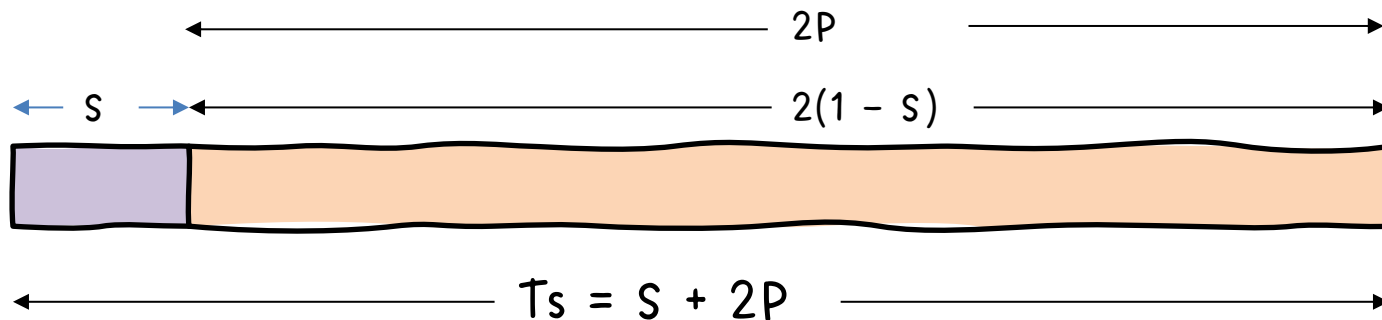
ธรรมชาติของการประมวลผลแบบขนาน (parallel)

- จากกฎของ Amdahl การเพิ่มจำนวน processors ไม่ได้ช่วยให้ประมวลผลงานได้เร็วขึ้นได้เสมอไป
- กฎนี้ทำให้ จำนวน processors ที่ถูกพัฒนาขึ้นใน parallel computers ในช่วงปี 1980 – 1990 มีจำนวน processor ไม่เกิน 256 processors
- ทำไมในปัจจุบัน Supercomputer ถึงมีจำนวน processors เป็น แสน processors และทำไมจำนวน cores บน CPU ระบบ server เพิ่มขึ้นตลอด
- ในช่วงต้นทศวรรษ 1990 มี Gustafson's law เกิดขึ้นและเกิดมาตรวัดใหม่ เรียกว่า Scalability
- หลักการคือ “จะเพิ่ม processor ก็ต้องเพิ่มขนาดของปัญหาด้วย ถึงจะคุ้ม”
- นศ ไม่เพิ่มคนงานจาก 10 คน เป็น 1000 เพื่อสร้างบ้านหลังเดียว แต่เพิ่มขึ้นให้สร้างหมู่บ้านจัดสรรที่มีบ้านเป็น หลายสิบหรือหลายร้อยหลัง
- https://en.wikipedia.org/wiki/Gustafson%27s_law

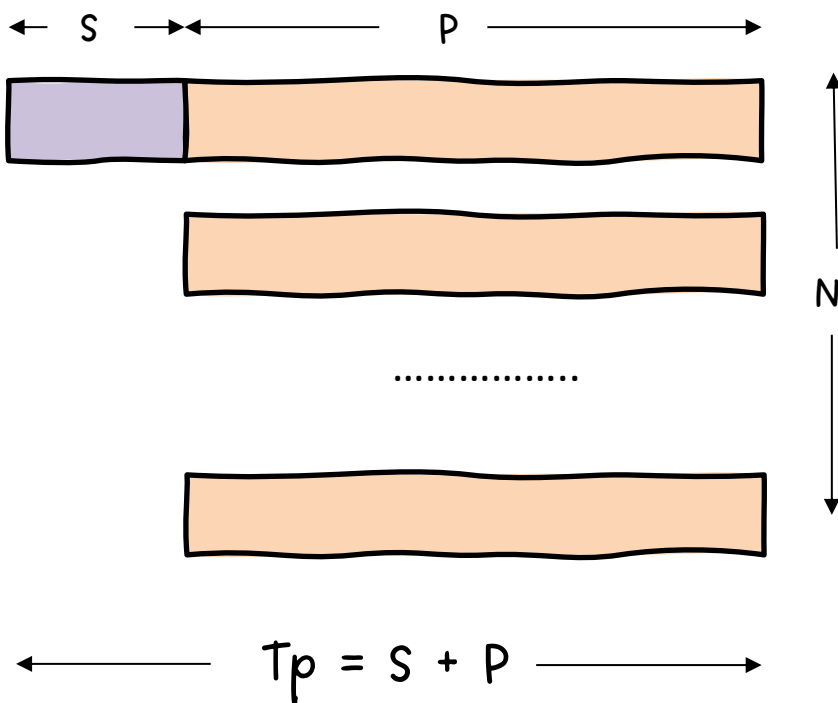
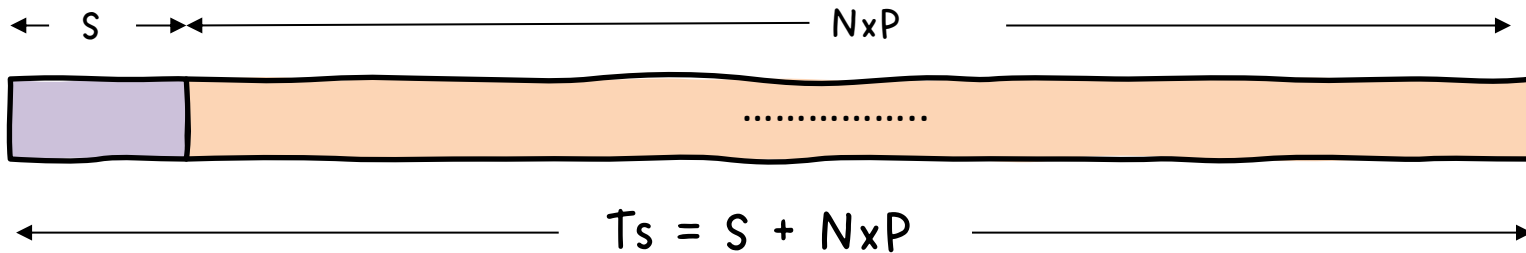
Gustafson's Law

- สมมติว่าเวลาที่ใช้ประมวลผลแบบ sequential คือ 1
- สมมติว่างานที่แบ่งไม่ได้คือ S
- ให้ $P = (1 - S)$

- $Work = 1$
- S is serial portion
- N processing cores



Gustafson's Law

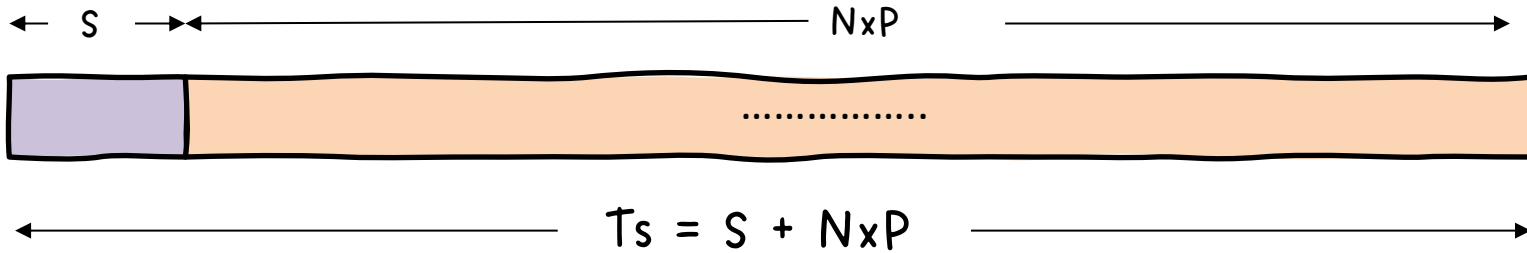


- Speedup $S(N)$ เมื่อใช้ N processors แล้วเร็วขึ้นกี่เท่า
- $S(N) = T_s / T_p$

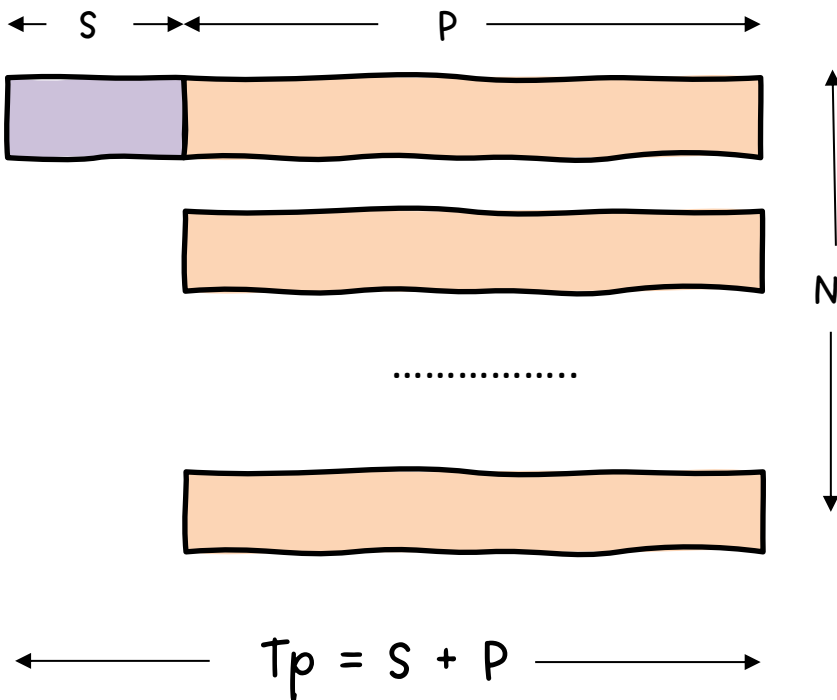
$$= \frac{S + NP}{S + P} = \frac{S + NP}{S + (1 - S)}$$

$$= S + NP$$
- เมื่อ N สูง $S(N)$ สูงตามแบบ linear
 — (P เป็น slope)

Efficiency



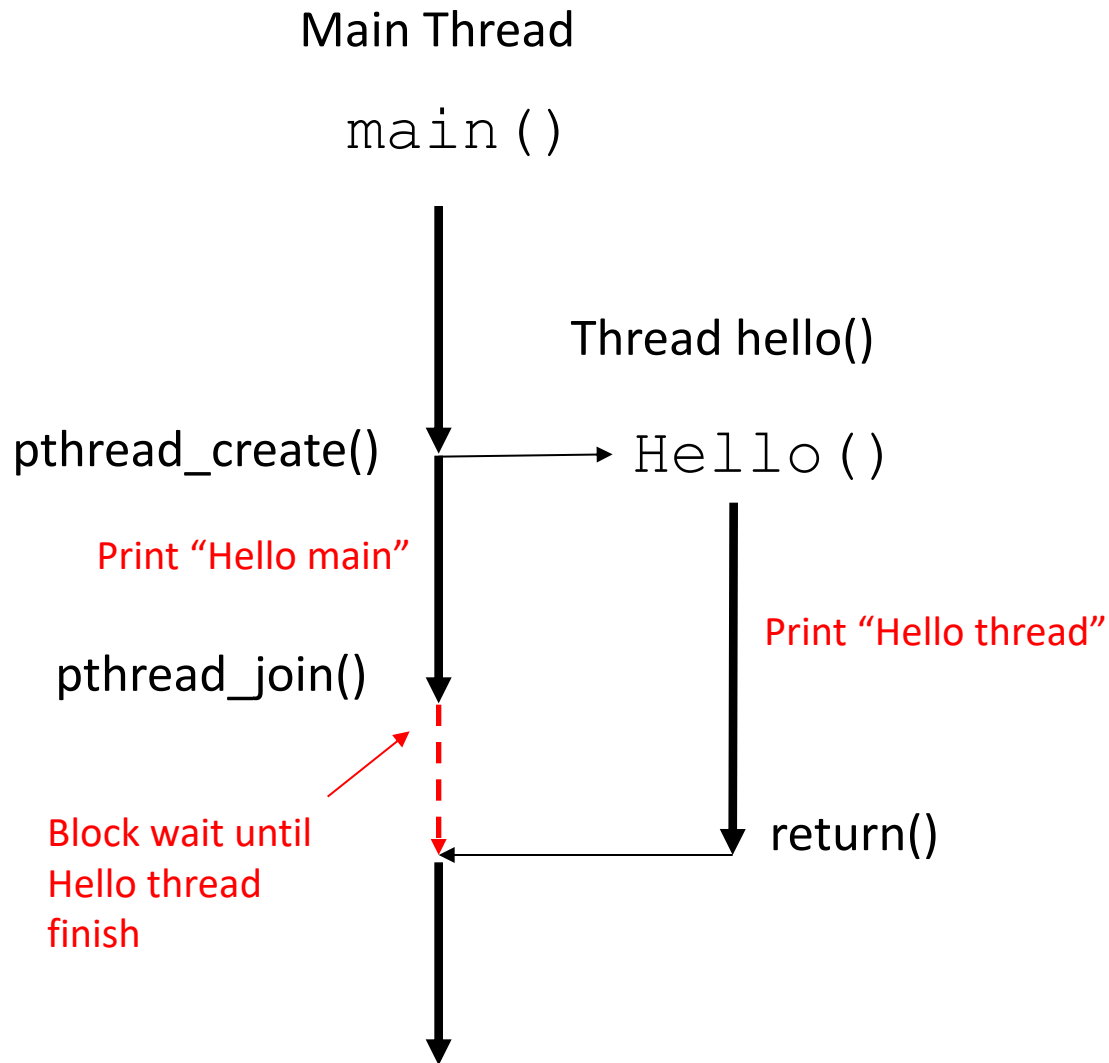
- $E(N) = S(N) / N$
- 1 is ideal
- > 1 is great!



Thread programming

- ภาษา C มี Library สำหรับสร้าง Thread เรียกว่า pthread library
- เราใช้คำสั่ง pthread_create เพื่อสร้างเธรด
- เราใช้คำสั่ง pthread_join เพื่อรอให้เธรดใดเธรดหนึ่งในโปรแกรมจบการประมวลผล
- เราใช้คำสั่ง pthread_exit เพื่อจบการประมวลผลของเธรด
- เราใช้คำสั่ง pthread_detach เพื่อประกาศว่าเธรดใดเธรดหนึ่งเป็นอิสระจะจบไปเองและไม่ต้องใช้ pthread_join รอ

./pth_hello1



Thread Creation

- Create a thread

```
#include <pthread.h>
```

```
int pthread_create(pthread_t *restrict tidp,  
                  const pthread_attr_t *restrict attr,  
                  void *(*start_rtn)(void *), void *restrict arg);
```

Returns: 0 if OK, error number on failure

Thread Creation Parameters

- *arg* is a typeless pointer *that can point to a structure if want to pass more than one args*
- The **pthread** functions usually return an error code when they fail

Thread Termination

A single thread can exit in three ways, thereby stopping its flow of control, without terminating the entire process.

- 1.** The thread can simply return from the start routine. The return value is the thread's exit code.
- 2.** The thread can be canceled by another thread in the same process.
- 3.** The thread can call `pthread_exit`.

Thread Termination

```
#include <pthread.h>
```

```
void pthread_exit(void *rval_ptr);
```

- The *rval_ptr pointer* will be available to other threads by calling the `pthread_join()` function

pthread_join()

```
#include <pthread.h>
```

```
int pthread_join(pthread_t thread, void **rval_ptr);
```

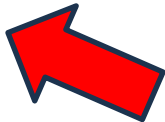
Returns: 0 if OK, error number on failure

- The calling thread will block until the specified thread
 - calls pthread_exit,
 - returns from its start routine, or
 - is canceled.
- If the thread simply returned from its start routine, **rval_ptr** will contain the return code.

- If the **thread was canceled**, the memory location specified by `rval_ptr` is set to `PTHREAD_CANCELED`.
- By calling `pthread_join`, we automatically place the thread with which we're joining in the **detached state** so that its resources can be recovered (เมื่อเทรดนั้นจบการประมวล).
- Return If the thread was already in the detached state, `pthread_join` can failing `EINVAL`
- If `rval_ptr == NULL`, `pthread_join` will wait but not return status info

example1.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <pthread.h>
5
6 void * foo(void * arg){
7     printf("Hello World\n");
8     return(NULL);
9 }
10
11 int main(void){
12
13     pthread_t mtid, ntid;
14
15     mtid = pthread_self();
16     pthread_create(&ntid, NULL, foo, NULL);
17
18     printf("main thread id = %lu foo thread id = %lu\n",
19           (unsigned long) mtid,
20           (unsigned long) ntid);
21
22 }
```



ถ้า main จบก่อน เทรด foo ก็จะไม่เห็นผลลัพธ์ของ foo

example2.c

```
#include <pthread.h>

pthread_t ntid;

void
printids(const char *s)
{
    pid_t      pid;
    pthread_t   tid;

    pid = getpid();
    tid = pthread_self();
    printf("%s pid %lu tid %lu (0x%lx)\n", s, (unsigned long)pid,
        (unsigned long)tid, (unsigned long)tid);
}

void *
thr_fn(void *arg)
{
    printids("new thread: ");
    return((void *)0);
}
```

```
int
main(void)
{
    int    err;

    -----

    err = pthread_create(&ntid, NULL, thr_fn, NULL);
    if (err != 0)
        err_exit(err, "can't create thread");
    printids("main thread:");
    sleep(1);
    exit(0);
}
```

example3.c

```
#include <pthread.h>

void *
thr_fn1(void *arg)
{
    printf("thread 1 returning\n");
    return((void *)1);
}

void *
thr_fn2(void *arg)
{
    printf("thread 2 exiting\n");
    pthread_exit((void *)2);
}

int
main(void)
{
    int          err;
    pthread_t    tid1, tid2;
    void         *tret;

    err = pthread_create(&tid1, NULL, thr_fn1, NULL);
    if (err != 0)
        err_exit(err, "can't create thread 1");
    err = pthread_create(&tid2, NULL, thr_fn2, NULL);
```

```
if (err != 0)
    err_exit(err, "can't create thread 2");
err = pthread_join(tid1, &tret);
if (err != 0)
    err_exit(err, "can't join with thread 1");
printf("thread 1 exit code %ld\n", (long)tret);
err = pthread_join(tid2, &tret);
if (err != 0)
    err_exit(err, "can't join with thread 2");
printf("thread 2 exit code %ld\n", (long)tret);
exit(0);
}
```

```
$ ./a.out
```

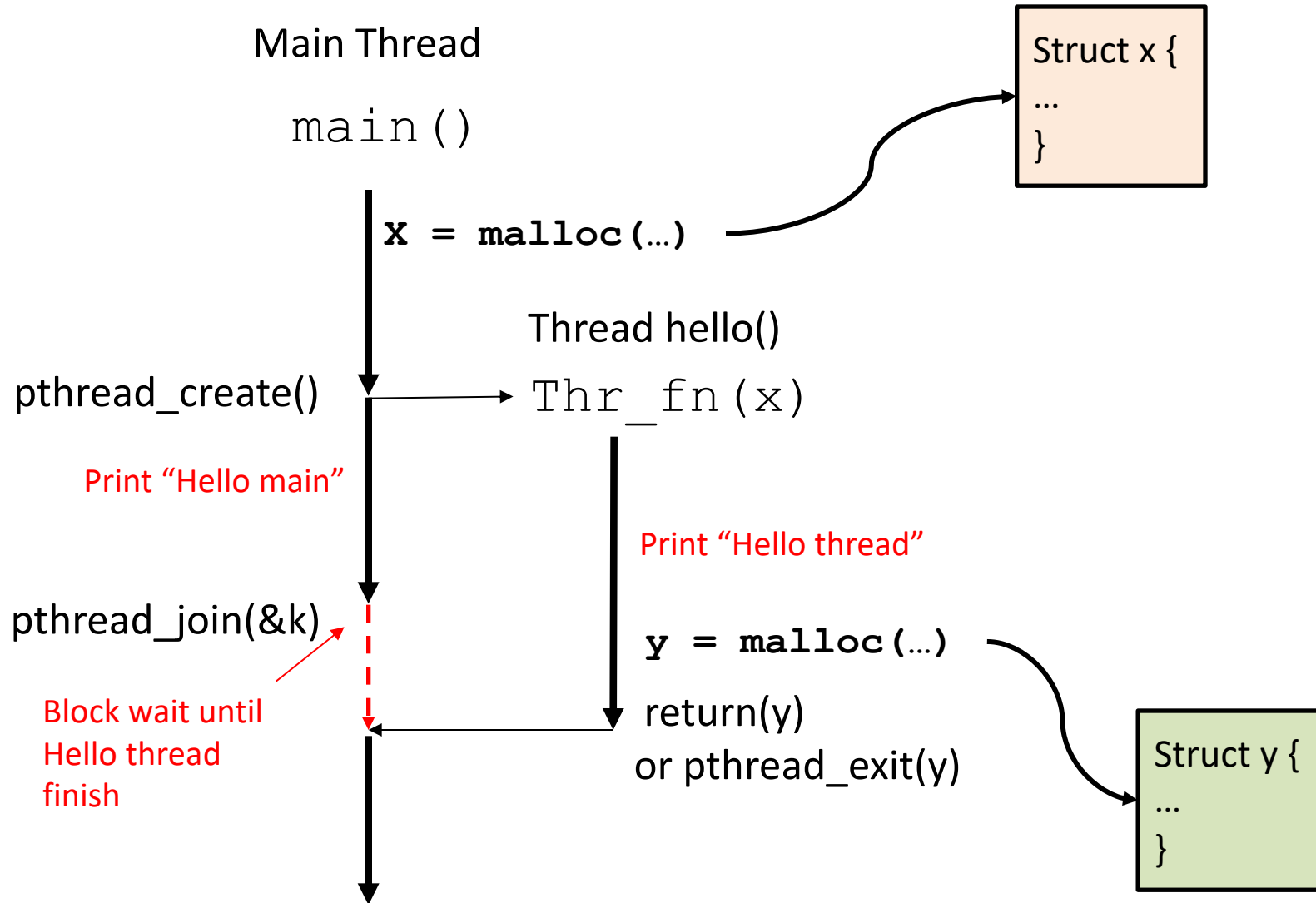
```
thread 1 returning
```

```
thread 2 exiting
```

```
thread 1 exit code 1
```

```
thread 2 exit code 2
```

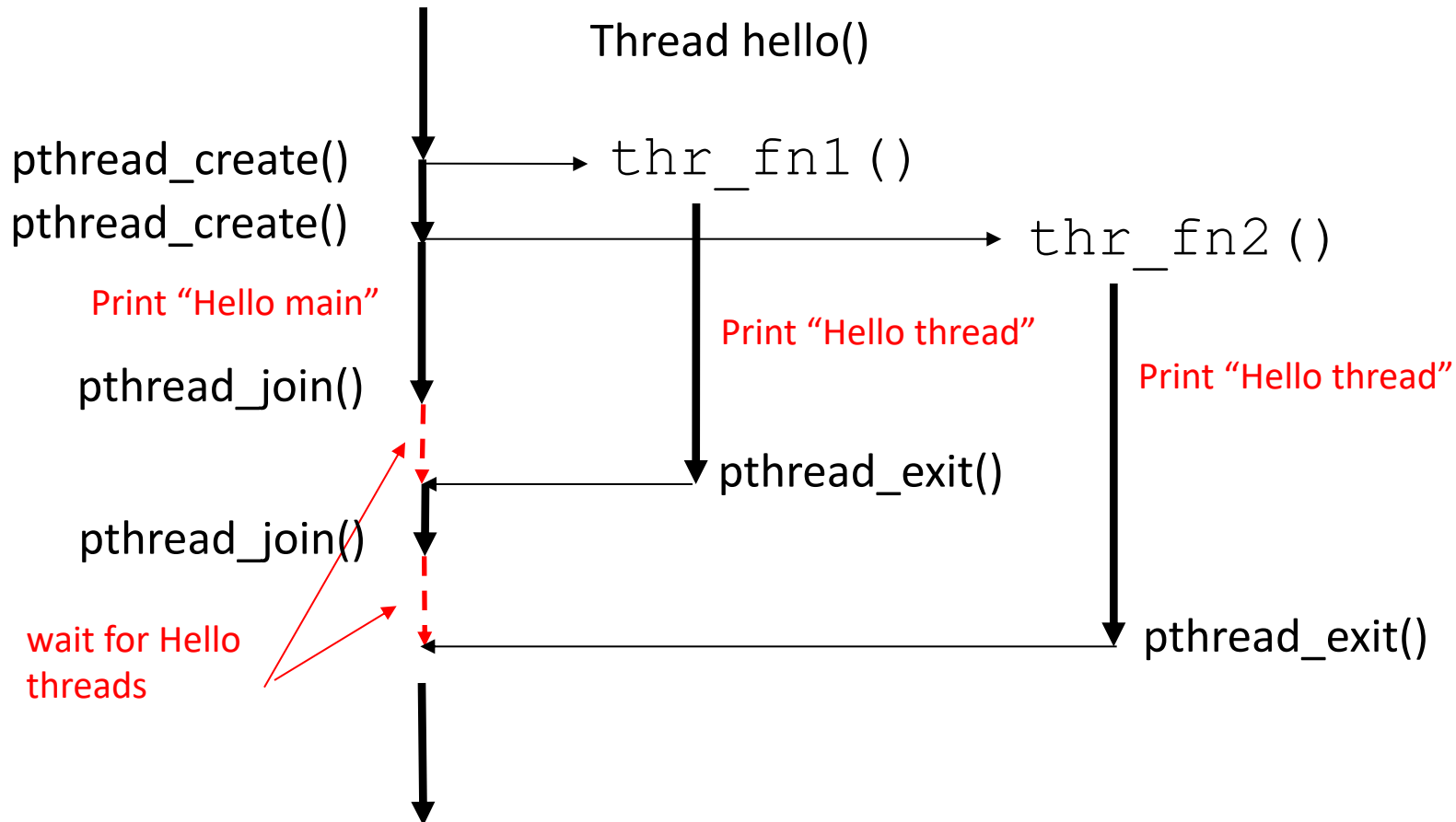
./example4



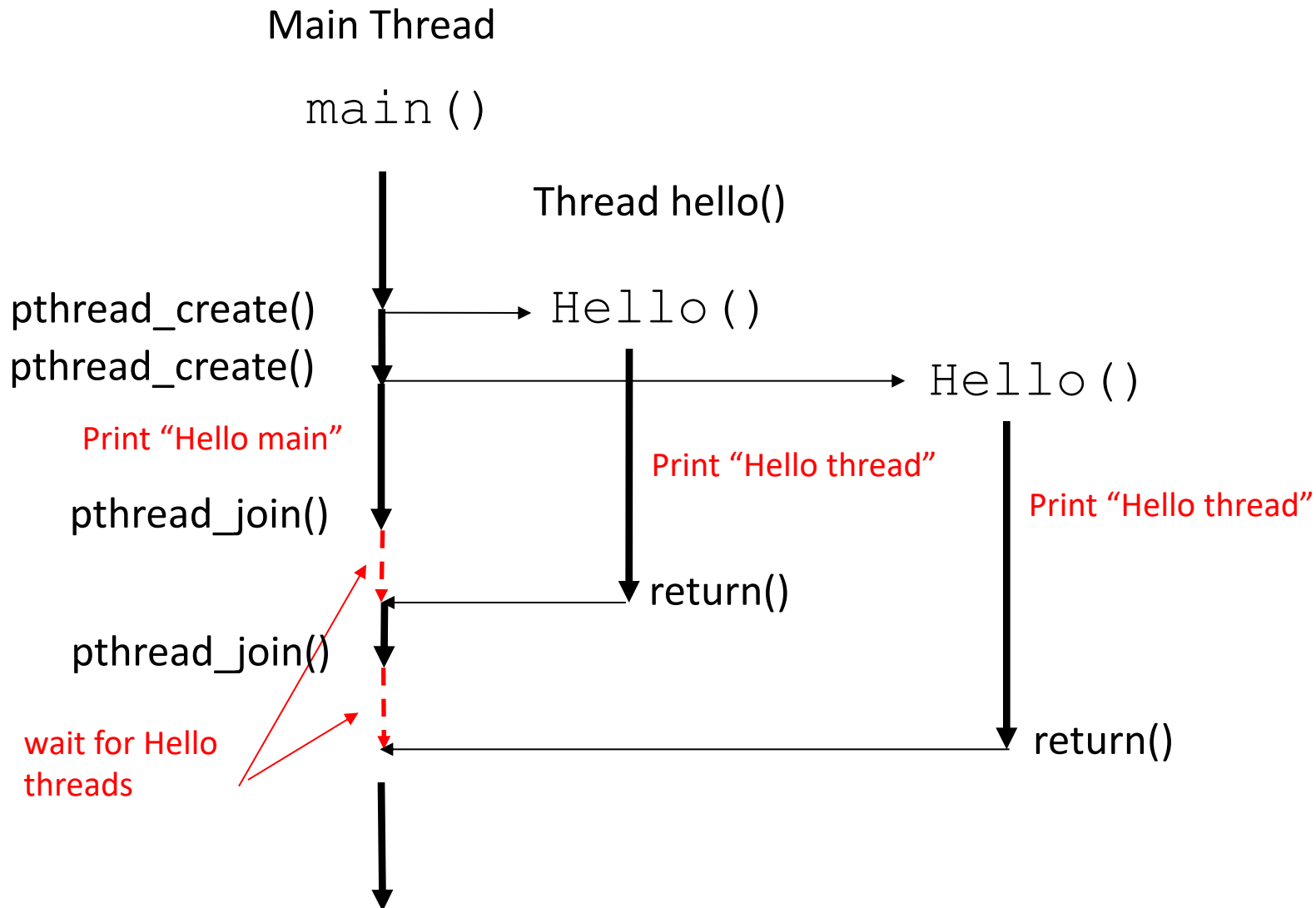
./a.out

Main Thread

main()



ตัวอย่าง ./hello 2



Hello World! (1)

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

/* Global variable: accessible to all threads */
int thread_count;

void *Hello(void* rank); /* Thread function */

int main(int argc, char* argv[]) {
    long          thread; /* Use long in case of a 64-bit system */
    pthread_t* thread_handles;

    /* Get number of threads from command line */
    thread_count = strtol(argv[1], NULL, 10);

    thread_handles = malloc (thread_count*sizeof(pthread_t));
```

declares the various Pthreads functions, constants, types, etc.

Allocate memory to store thread handle objects.

Hello World! (2)

Repeatedly create a number of threads. Each running a “Hello()” function

```
for (thread = 0; thread < thread_count; thread++)  
    pthread_create(&thread_handles[thread], NULL,  
        Hello, (void*) thread);
```

```
printf("Hello from the main thread\n");
```

```
for (thread = 0; thread < thread_count; thread++)  
    pthread_join(thread_handles[thread], NULL);
```

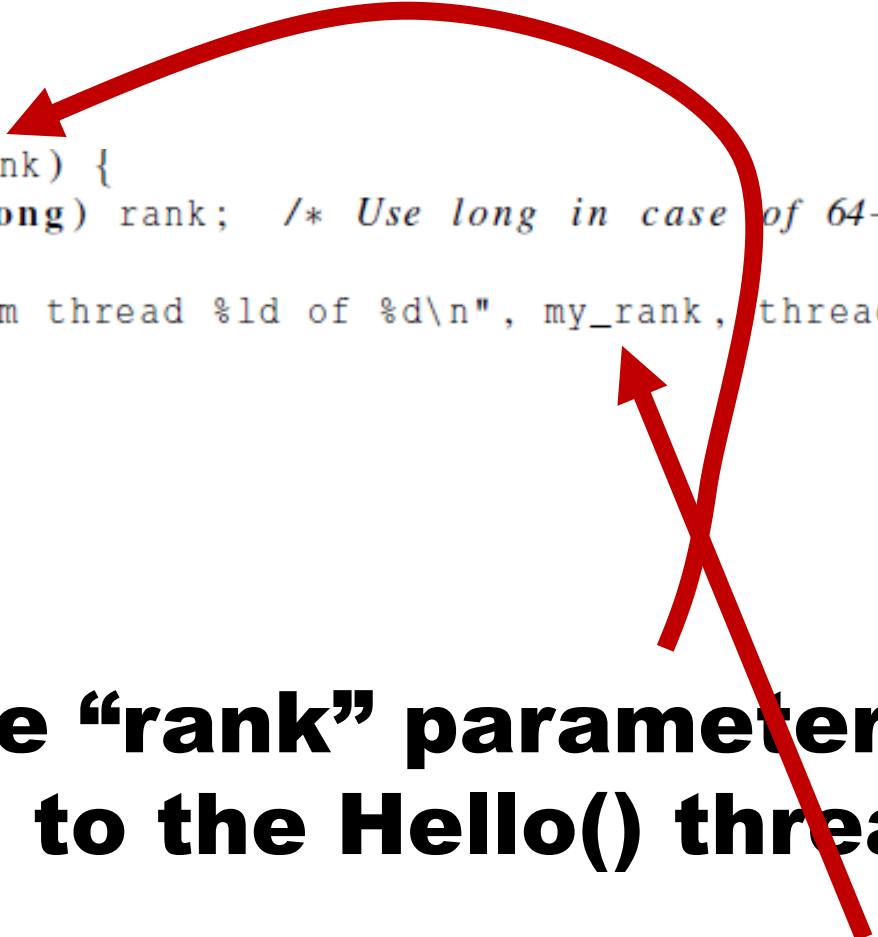
```
free(thread_handles);  
return 0;
```

```
} /* main */
```

Main thread call “pthread_join” to wait until every thread finishes.

Hello World! (3)

```
void *Hello(void* rank) {  
    long my_rank = (long) rank;  /* Use long in case of 64-bit system */  
  
    printf("Hello from thread %ld of %d\n", my_rank, thread_count);  
  
    return NULL;  
} /* Hello */
```



Why the “rank” parameter is passed to the Hello() thread?

Programmer want to use it here.

pthread_detach()

```
#include <pthread.h>
```

```
int pthread_detach(pthread_t tid);
```

Returns: 0 if OK, error number on failure

- We can detach a thread by calling pthread_detach.
- After a thread is detached, *we cannot* use the pthread_join function to wait for its termination status, because calling pthread_join for a detached thread results in undefined behavior.

process vs thread interfaces

Process primitive	Thread primitive	Description
fork	pthread_create	create a new flow of control
exit	pthread_exit	exit from an existing flow of control
waitpid	pthread_join	get exit status from flow of control
atexit	pthread_cleanup_push	register function to be called at exit from flow of control
getpid	pthread_self	get ID for flow of control
abort	pthread_cancel	request abnormal termination of flow of control

Figure 11.6 Comparison of process and thread primitives

a_{00}	a_{01}	$\cdots$	$a_{0,n-1}$
a_{10}	a_{11}	$\cdots$	$a_{1,n-1}$
$\vdots$	$\vdots$		$\vdots$
a_{i0}	a_{i1}	$\cdots$	$a_{i,n-1}$
$\vdots$	$\vdots$		$\vdots$
$a_{m-1,0}$	$a_{m-1,1}$	$\cdots$	$a_{m-1,n-1}$

$\begin{matrix} x_0 \\ x_1 \\ \vdots \\ x_{n-1} \end{matrix} =$

y_0
y_1
$\vdots$
$y_i = a_{i0}x_0 + a_{i1}x_1 + \cdots a_{i,n-1}x_{n-1}$
$\vdots$
y_{m-1}

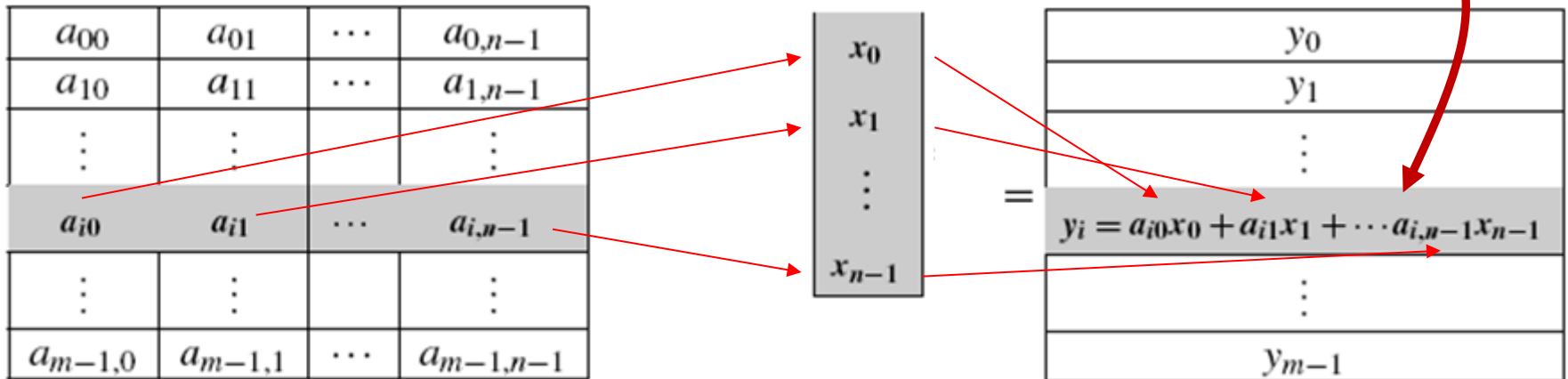
MATRIX-VECTOR MULTIPLICATION IN PTHREADS

Serial pseudo-code

$$y_i = \sum_{j=0}^{n-1} a_{ij}x_j$$

```

/* For each row of A */
for (i = 0; i < m; i++) {
    y[i] = 0.0;
    /* For each element of the row and each element of x */
    for (j = 0; j < n; j++)
        y[i] += A[i][j]* x[j];
}
    
```



Divide the problem

$$y_i = \sum_{j=0}^{n-1} a_{ij}x_j$$

a00 a01 a02 a03 a04 a05

x0

y0

a10 a11 a12 a13 a14 a15

x1

y1

a20 a21 a22 a23 a24 a25

x2

y2

a30 a31 a32 a33 a34 a35

x3

y3

a40 a41 a42 a43 a44 a45

x4

y4

a50 a51 a52 a53 a54 a55

x5

y5

Thread 0

a00 a01 a02 a03 a04 a05

x0

y0

a10 a11 a12 a13 a14 a15

x1

y1

x2

x3

x4

x5

Thread 0

$$y_0 = a_0 \cdot x$$

a00 a01 a02 a03 a04 a05

a10 a11 a12 a13 a14 a15

x0

x1

x2

x3

x4

x5

y0

y1

Thread 0

$$y1 = a1 \cdot x$$

a00 a01 a02 a03 a04 a05

a10 a11 a12 a13 a14 a15

x0

x1

x2

x3

x4

x5

y0

y1

Thread 1

$$y_2 = a_2 \cdot x$$
$$y_3 = a_3 \cdot x$$

x0

x1

a20 a21 a22 a23 a24 a25

x2

y2

a30 a31 a32 a33 a34 a35

x3

y3

x4

x5

Thread 2

$$y4 = a4 \cdot x$$
$$y5 = a5 \cdot x$$

x0

x1

x2

x3

a40 a41 a42 a43 a44 a45

x4

y4

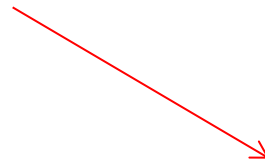
a50 a51 a52 a53 a54 a55

x5

y5

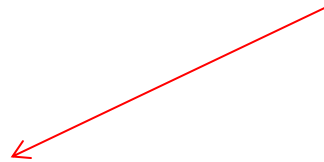
Using 3 Pthreads

Thread	Components of y
0	y[0], y[1]
1	y[2], y[3]
2	y[4], y[5]



thread 0

```
y[0] = 0.0;  
for (j = 0; j < n; j++)  
    y[0] += A[0][j]*x[j];
```



general case

```
y[i] = 0.0;  
for (j = 0; j < n; j++)  
    y[i] += A[i][j]*x[j];
```


Pthreads matrix-vector multiplication

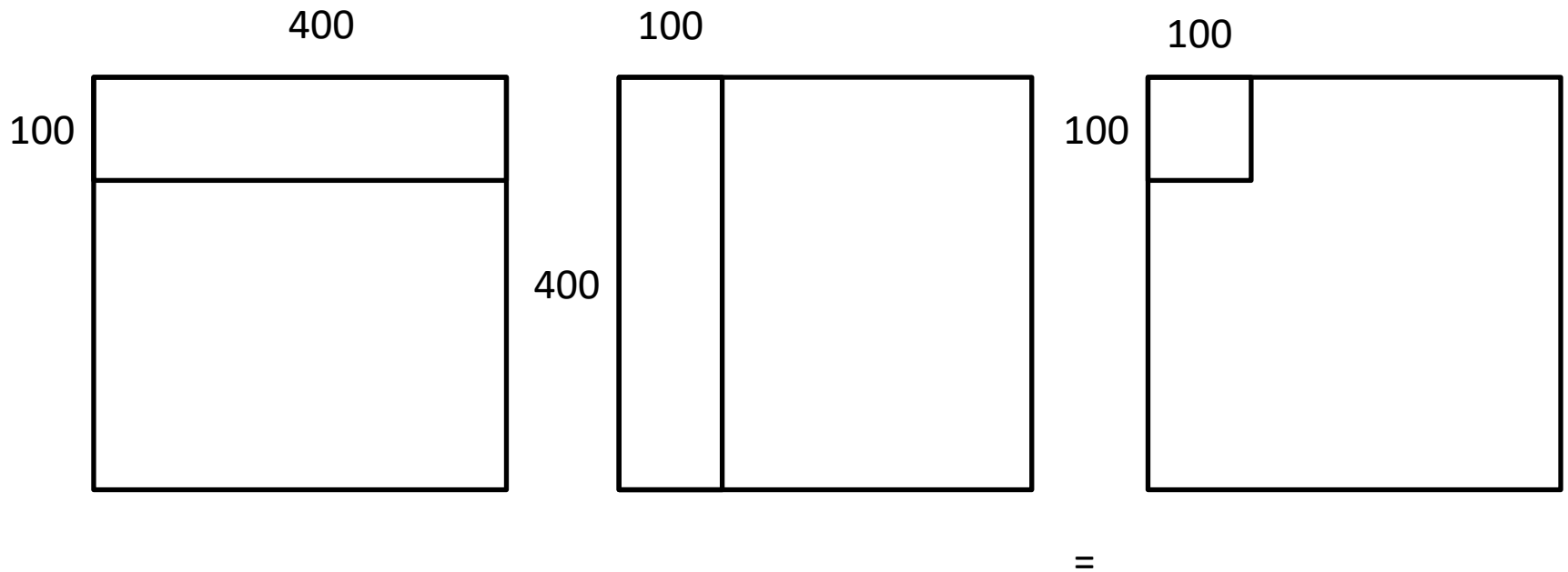
```
void *Pth_mat_vect(void* rank) {  
    long my_rank = (long) rank;  
    int i, j;  
    int local_m = m/thread_count;  
    int my_first_row = my_rank*local_m;  
    int my_last_row = (my_rank+1)*local_m - 1;
```

rank	my_first_row	my_last_row
0	$0 * (6/3) = 0$	$(0+1) * (6/3) - 1 = 1$
1	$1 * (6/3) = 2$	$(1+1) * (6/3) - 1 = 3$
2	$2 * (6/3) = 4$	$(2+1) * (6/3) - 1 = 5$

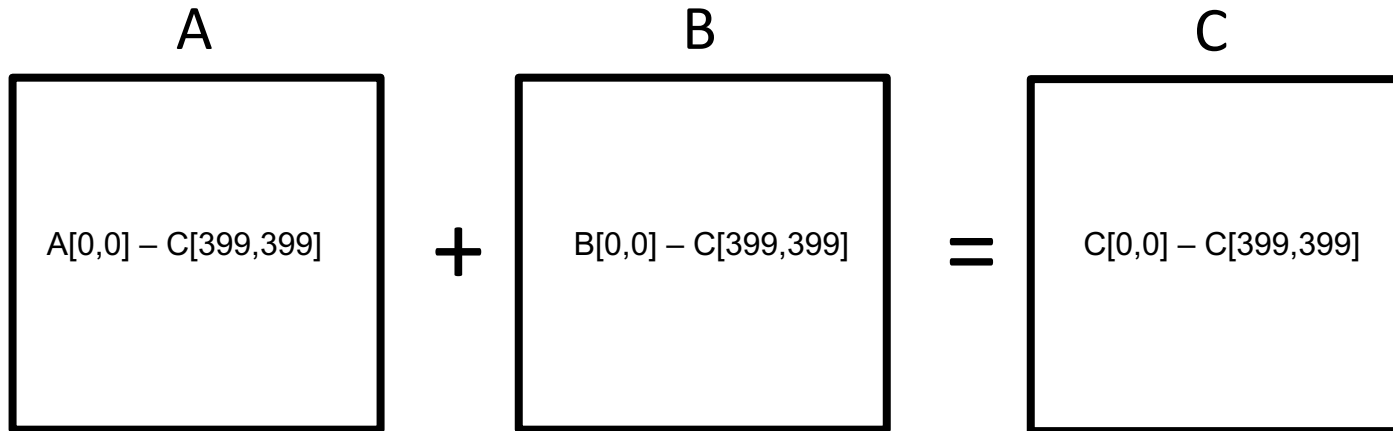
```
    return NULL;  
} /* Pth_mat_vect */
```

ตัวอย่าง คำถาม 1

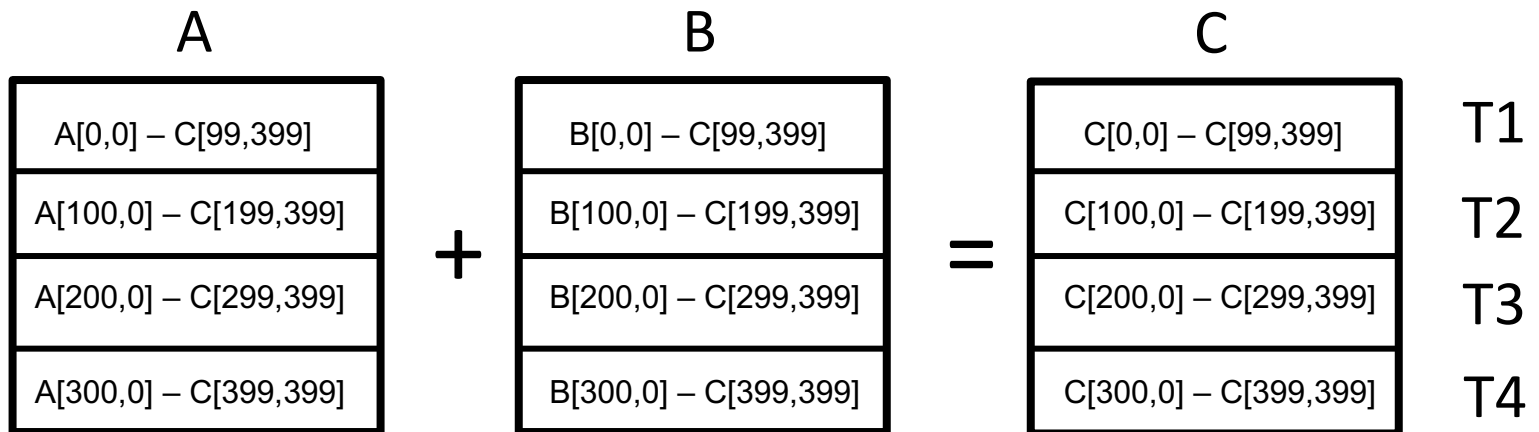
$$A_{(0..99)(0..399)} \times B_{(0..399)(0..99)} = D_{(0..99)(0..99)}$$



ตัวอย่าง คำถาม 2

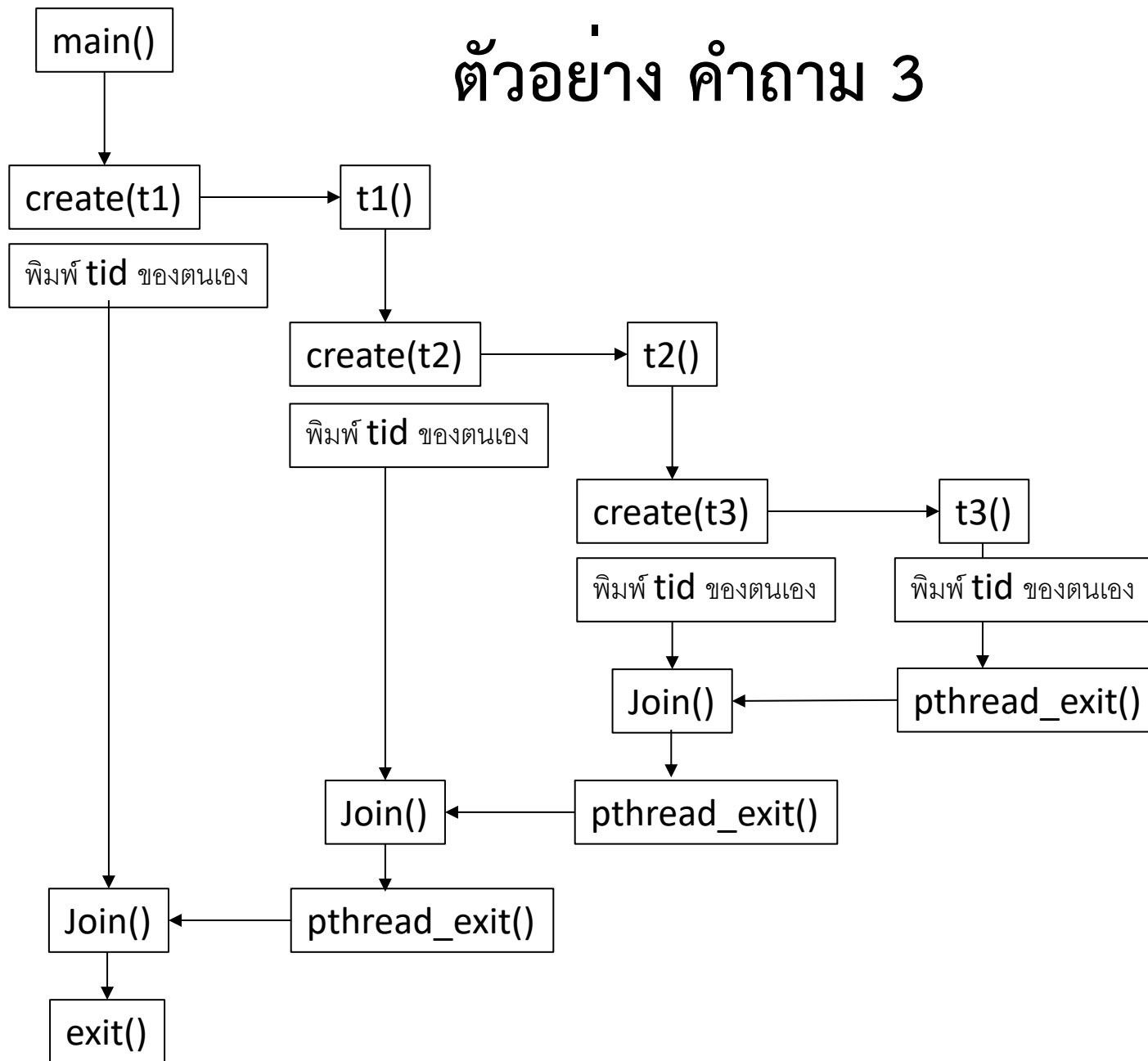


(ก)



(ข)

ตัวอย่าง คำถาม 3



Synchronization

- อ่าน บทที่ 6 7 และ 8 ใน “Operating System Concepts” Text book
 - บทที่ 6 Synchronization Tools
 - บทที่ 7 Synchronization Examples
 - บทที่ 8 Deadlock
- ใน Slide จะ cover เพียงแค่ subset ของเนื้อหาใน Text book เท่านั้น
- Slide จะ Focus ที่ Pthread Programming

ความไม่รู้อนาคตในการแข่งขันกัน

- เวลาที่มีการแข่งขันกันถึงแม้ว่าเราคิดว่าคนหนึ่งนำอยู่แต่ผลการแข่งขันในอนาคตอาจเป็นอย่างไรก็ได้
- เนื่องจาก ความไม่แน่นอนที่อาจเกิดขึ้นได้กับผู้แข่งขันคนใดคนหนึ่ง
- เช่นตัวอย่างข้างล่าง เราจะบอกได้ไหมว่า Thread1 จะจบก่อน Thread2 ถ้าทั้งสองรันบนระบบ multiprogramming

Thread 1	Thread 2
.... Printf(abc)	 Sleep(10) Printf(abc)

Race Condition

- เทรดทุกเทรดใช้ memory ร่วมกัน
- การ assign ค่าให้กับ variable ในภาษา C ประกอบไปด้วยคำสั่งภาษาเครื่องมากกว่า 1 คำสั่ง
- Race Condition คือการที่เทรด(หรือโพรเซส)หลายเทรดเข้าถึงและเปลี่ยนแปลงข้อมูลพร้อมกันแล้วผลลัพธ์ของการเปลี่ยนแปลงนั้นไม่แน่นอน เนื่องจากไม่สามารถกำหนดได้ว่าเทรดจะเข้าถึงข้อมูลเมื่อใด
- จำเป็นต้องมีการ Synchronization ระหว่างสองเทรด

ปัญหา เขียนและอ่านพร้อมกัน

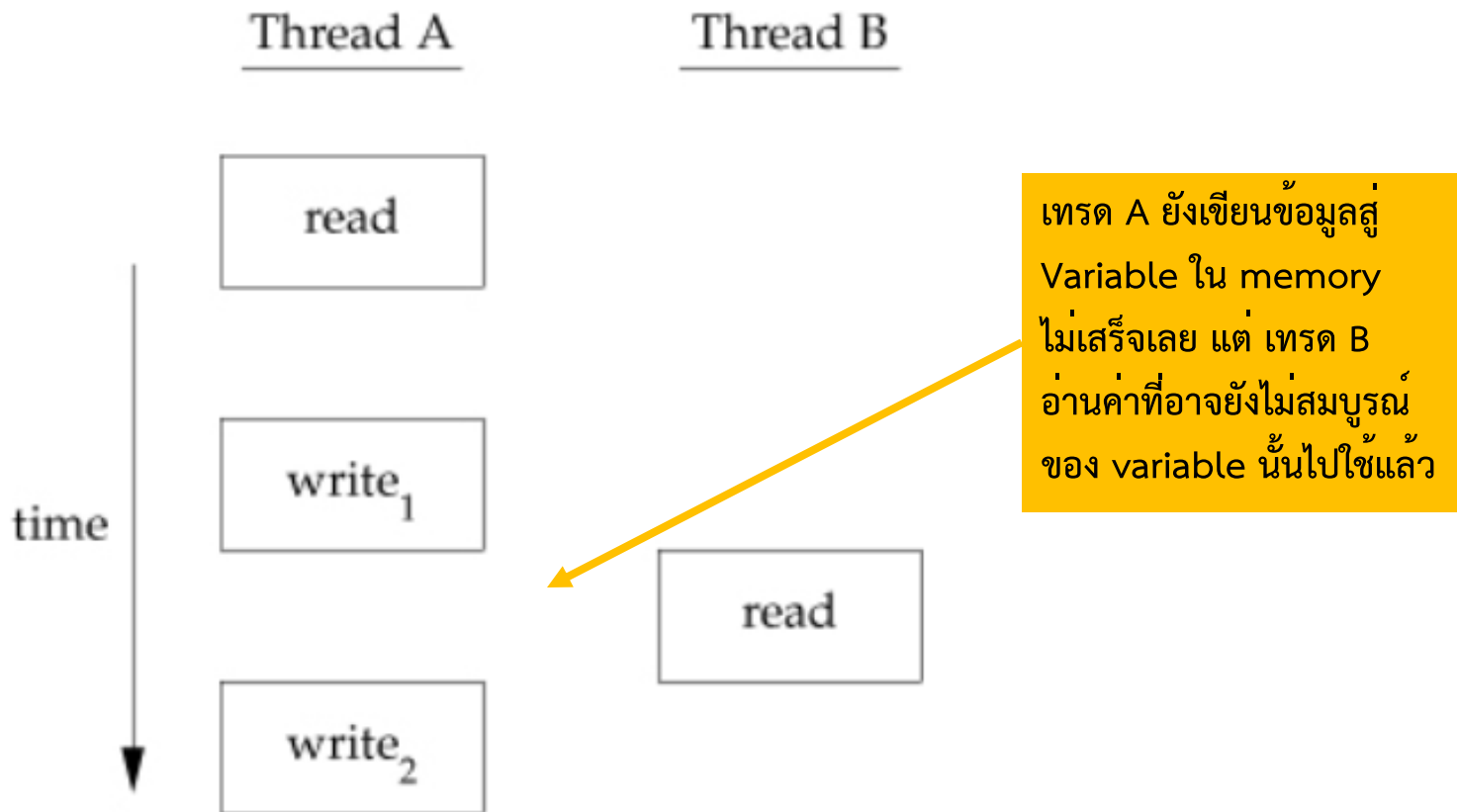


Figure 11.7 Interleaved memory cycles with two threads

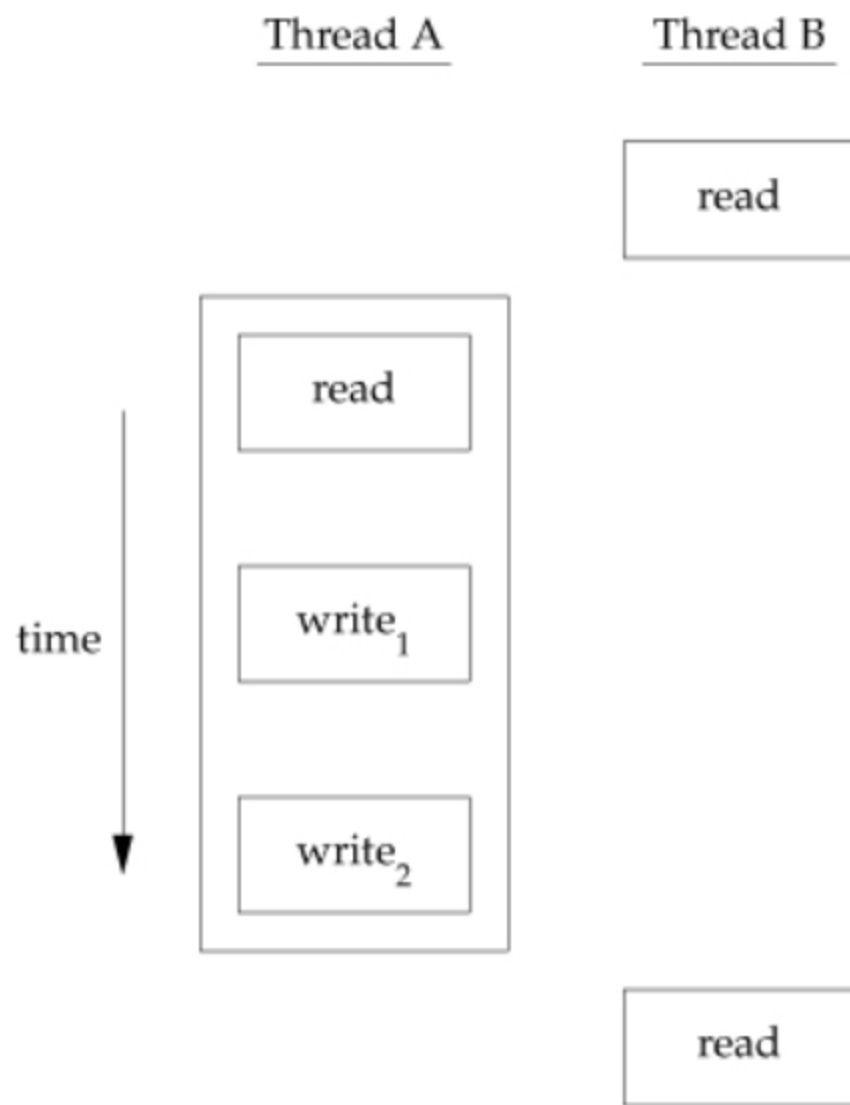
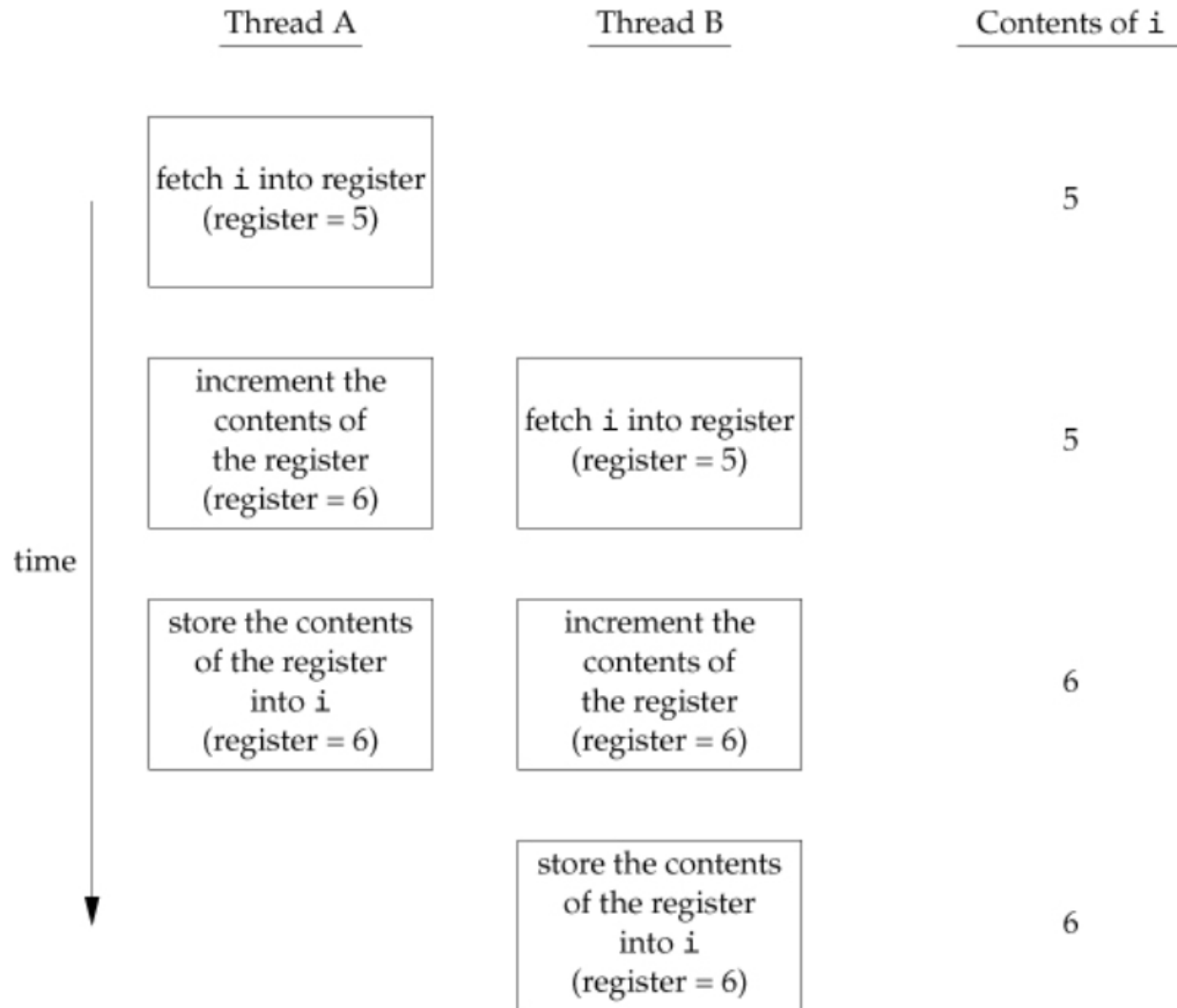


Figure 11.8 Two threads synchronizing memory access

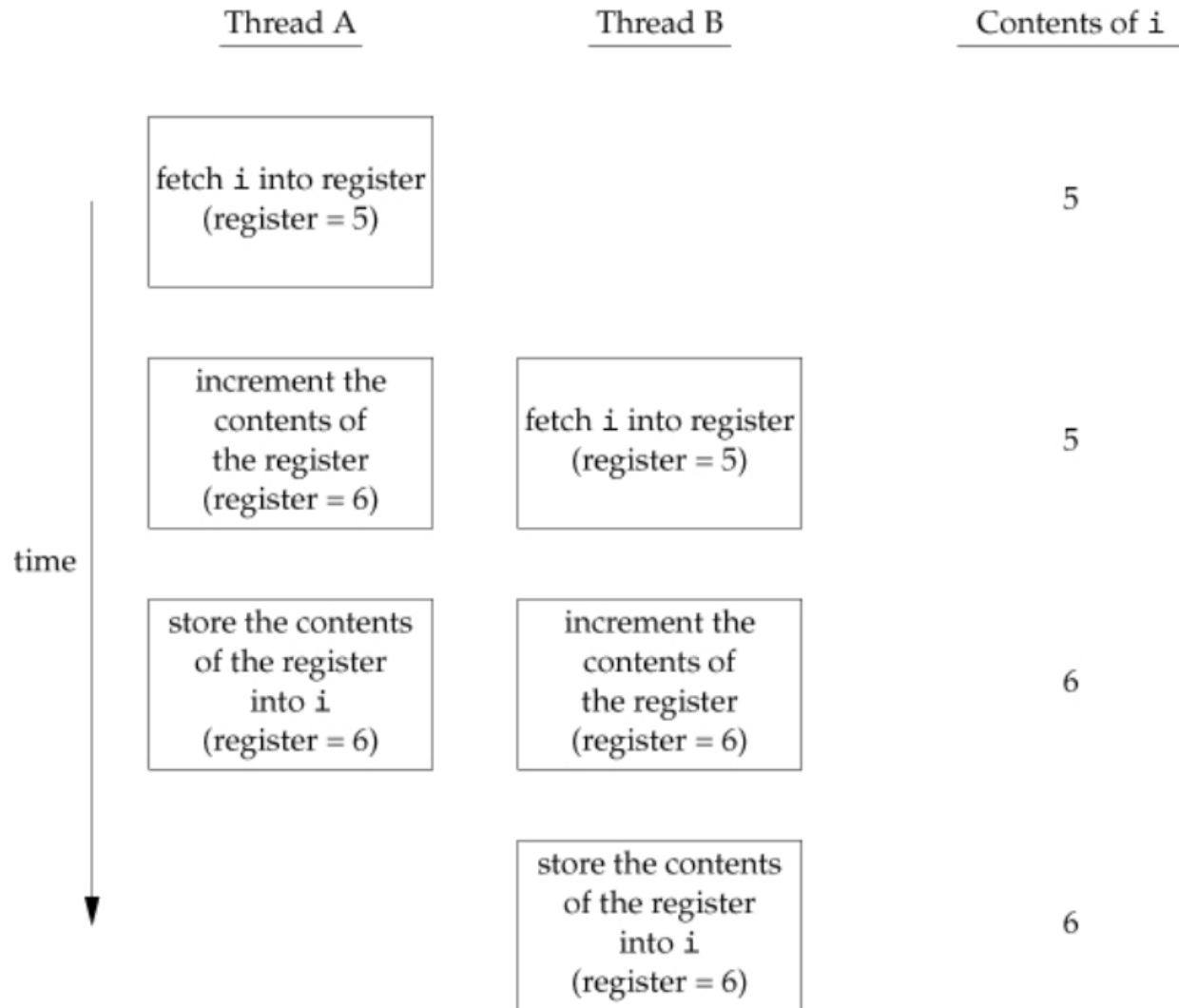
ปัญหาการ **เขียน** (update) ค่าพร้อมกัน

- อ่านค่าจากตัวแปร i มาที่ register
- เพิ่มค่าใน register + 1
- เขียนค่าจาก register สู่อตัวแปร i
- เกิดปัญหาเนื่องจากการ update **ไม่เป็น atomic** การ update เหลื่อมกัน



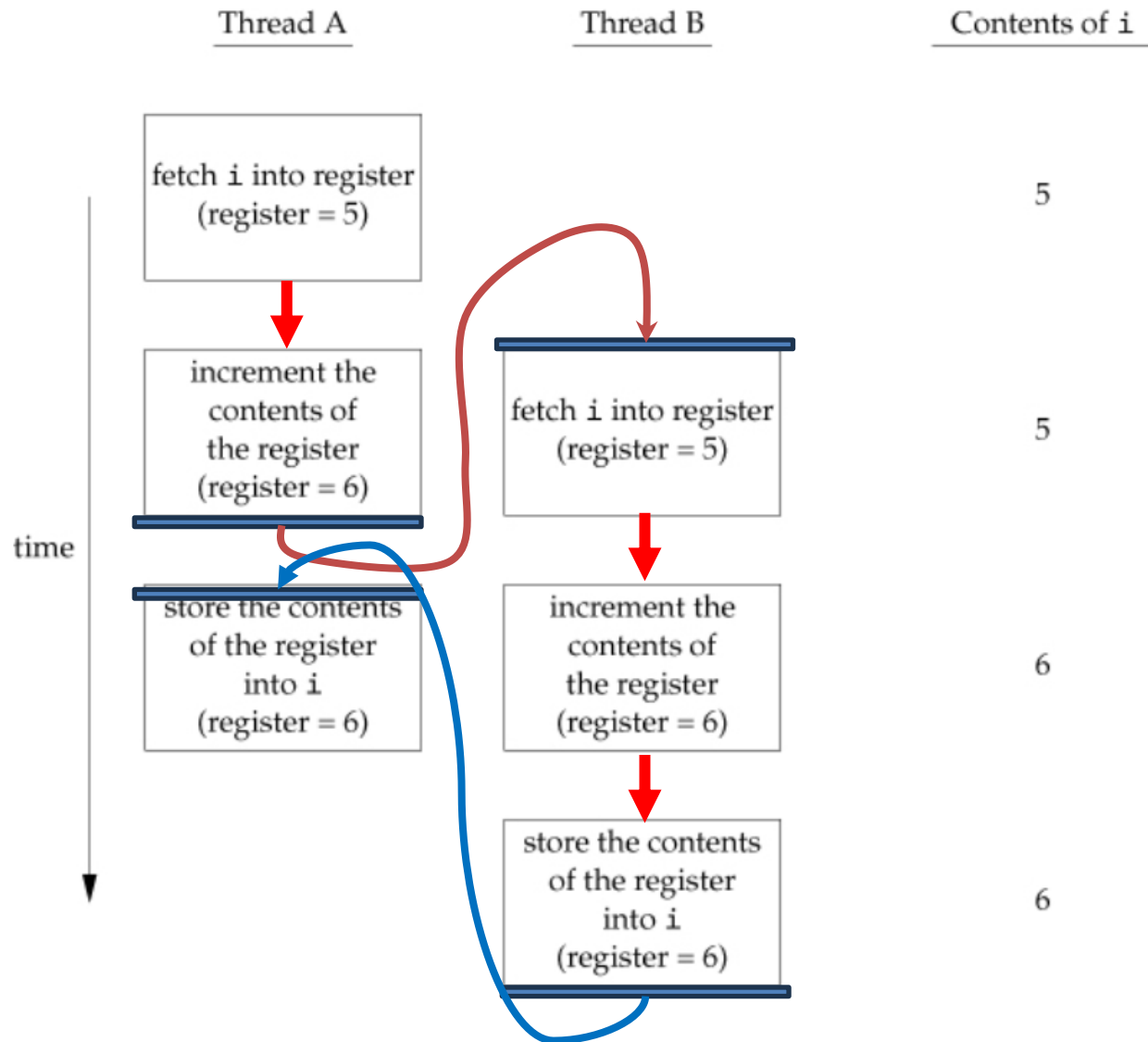
แบบ Multi-processing

- กรณี Thread A และ Thread B รันอยู่บนต่าง CPU cores



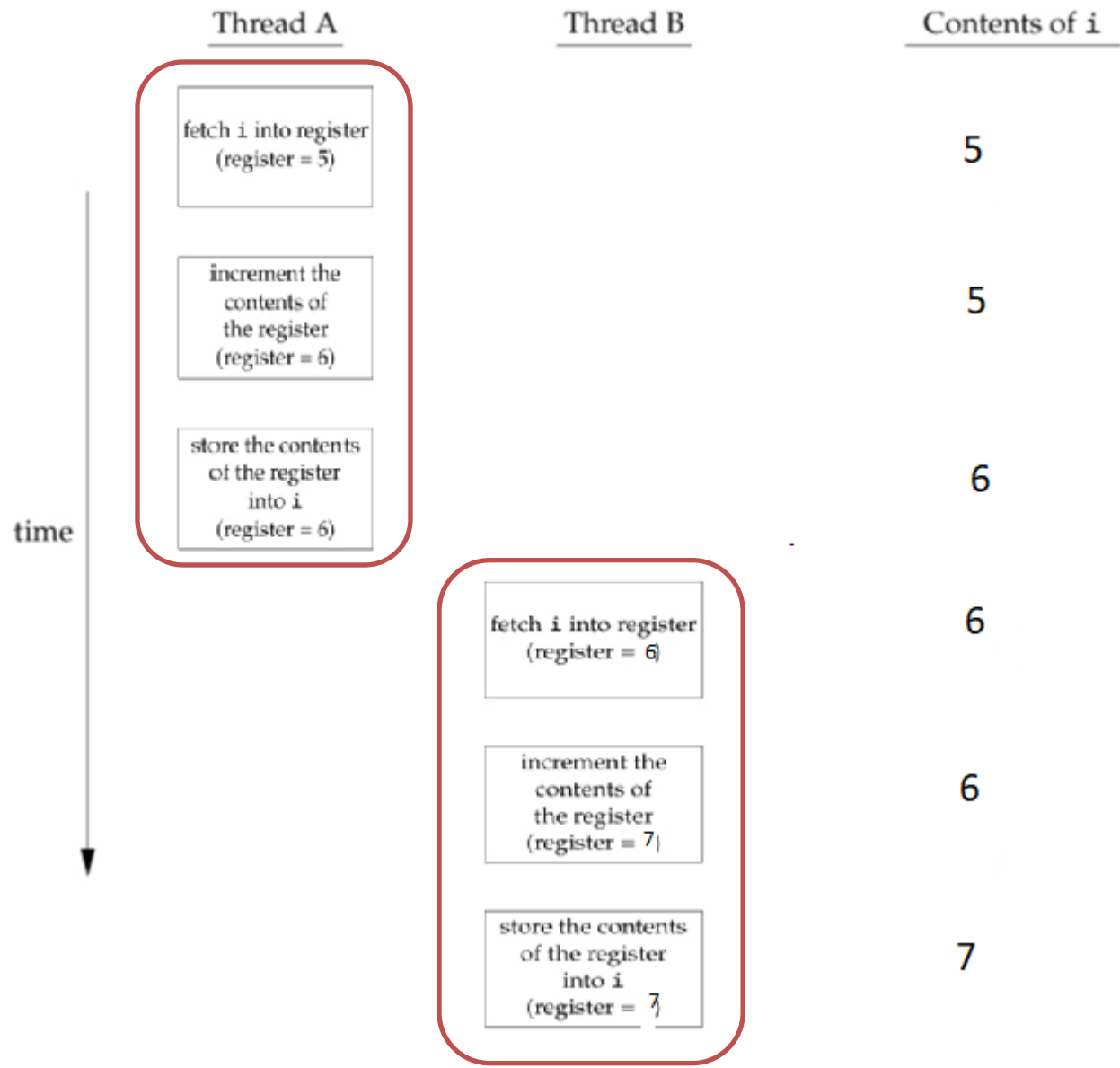
แบบ Mutiproramming

- กรณี Thread A และ Thread B รันอยู่บน CPU core เดียวกัน
- Thread A รันสอง Inst แล้วเกิด context switching
- CPU scheduling นั้น คาดเดาพฤติกรรมได้ยาก โดยเฉพาะในระบบที่ซับซ้อน
- Thread B รัน 3 Inst แล้วเกิด context switching กลับไปรัน A
- ผลลัพธ์คือค่า shared variable i ใน memory ผิด



Synchronization แก้ปัญหาการ update ค่าพร้อมกัน

- ต้อง synchronize ให้ Thread A update ก่อนแล้วตามด้วย B หรือ Thread B update ก่อนแล้วตามด้วย Thread A จึงจะถูก
- แบบนี้ A เข้าเปลี่ยนค่า i ก่อนหรือหลัง B ก็จะทำให้ผลลัพธ์เหมือนกัน



Critical Section

- ในระบบที่ประกอบไปด้วย n เทรด $\{P_0, P_1, \dots, P_{n-1}\}$.
- แต่ละเทรดต้องเข้าถึง ส่วนหนึ่งของ code ที่หลายเทรดใช้ ข้อมูลหรือตัวแปรบางอย่างร่วมกัน เราเรียก Code ส่วนนั้นของแต่ละเทรดว่า Critical Section
- คุณสมบัติ
 - Mutual Exclusion: เมื่อเทรดหนึ่งประมวลผลใน Critical Section ของตนแล้ว เทรดอื่นจะไม่สามารถเข้ามาประมวลผลใน Critical Section ของเขาได้ ณ. เวลาใดเวลาหนึ่ง มีเทรดเดียวเท่านั้นที่เข้าประมวลใน Critical Section ได้
 - Progress: อาจมีหลายเทรดรอเข้า critical section เทรดที่รอทุกเทรดต้องแข่งขันกัน และได้รับการพิจารณาให้เข้า critical section
 - Bound waiting: เทรดที่รอจะต้องได้เข้า Critical section ของเขาในที่สุด ถึงแม้ว่าจะมีเทรดอื่นมาแทรก การแทรกต้องมีขอบเขตและจำนวนจำกัด

Critical Section

- ระบบที่มี Progress ไม่มี Bound waiting เช่น ระบบที่ทุกเธรดผ่านเข้า critical section แต่มีบางเธรดถูกแขงตลอด ทั้งระบบมี Progress ความคืบหน้า แต่มีบางเธรดอดตาย Stravation
- ระบบที่มี Bound waiting แต่ไม่มี Progress ได้แก่ ระบบที่กำหนดไว้ชัดเจนว่า เธรด B จะเข้า Ctitical Section หลังจากเธรด A แต่ เธรด A มันแต่ทำอย่างอื่นไม่เข้า Critical Section ซะที่ เรียกว่า A ไม่มีความคืบหน้า

การแก้ปัญหา Critical Section

- การแก้ปัญหามีทั้งแบบ Software-based และ Hardware-based
 - Software-based ดู บทที่ 6 ใน Text book
- ในปัจจุบันใช้ Hardware-based เครื่องมือที่สำคัญได้แก่
 1. Memory Barrier หรือ Memory fencing เนื่องจาก ตอนที่คอมไพเลอร์ compile โปรแกรมมันอาจสลับชุดคำสั่งในโปรแกรมเพื่อให้สามารถใช้ซีพียูแบบ Super Scalar ซึ่งประมวลผลแบบ Instruction level parallelism ให้ได้มากที่สุด Memory barrier ป้องกันไม่ให้คอมไพเลอร์ สลับเอาคำสั่งที่อยู่ภายใน Critical section มาอยู่ภายนอก

การแก้ปัญหา Critical Section

1. Memory Barrier
2. Hardware Instruction ที่สามารถเปลี่ยนค่าตัวแปรในหน่วยความจำ หรือสลับค่าตัวแปรในหน่วยความจำแบบ atomic คือเมื่อซีพียู ประมวลผลคำสั่งนี้มันจะเปลี่ยนแปลงค่าให้เสร็จภายในคำสั่งเดียว (ไม่เหมือนการ update ตัวแปรธรรมดาที่อาจมีเทรดอื่นมาแทรกได้)
 - คำสั่ง test_and_set และ compare_and_swap

การแก้ปัญหา Critical Section

- เนื่องจากการใช้คำสั่ง Hardware สำหรับจัดการ Critical section มีความยุ่งยาก
- OS จึงให้บริการกลไก **Mutual Exclusion** เป็น **System call** เพื่อให้บริการจัดการ Critical section และกำหนดว่าเทรดหรือโพรเซสใดจะต้องเปลี่ยน State ไปเป็น Sleep state
 - Linux ใช้ `futex(FUTEX_WAIT)` และ `futex(FUTEX_WAKE)` system calls
 - `Pthread_mutex_lock()` และ `pthread_mutex_unlock()` เรียก `futex()` อีกต่อหนึ่ง
 - เมื่อเทรดเรียกใช้ `futex` เทรดนั้นจะมีสถานะเป็น `TASK_INTERRUPTIBLE` (หมายความว่า มันสามารถถูกขัดจังหวะด้วย `SIGNAL` ได้)

Semaphore

- Semaphore เป็นเครื่องมือ Synchronization สำหรับแก้ปัญหา Critical Section อีกแบบหนึ่งที่เสนอโดย Edsger Dijkstra
- มี Semaphore S คือตัวแปรแบบ Integer ที่ทำงานร่วมกับ atomic operation สองแบบคือ `wait()` และ `signal()`

```
wait(S) {  
    while (S <= 0)  
        ; // busy wait  
    S--;  
}
```

```
signal(S) {  
    S++;  
}
```

Semaphore

```
wait(S) {  
    while (S <= 0)  
        ; // busy wait  
    S--;  
}
```

```
signal(S) {  
    S++;  
}
```

- สมมุติ Sync = 0

P1	P2
S1; Signal(Sync);	Wait(Sync); S2;

- S1 รันก่อน S2 เสมอ

Monitor

- Monitor เป็นเครื่องมือ Synchronization ในรูปแบบของ Abstract Data Type (ADT)
- ADT คือ Model สำหรับการ encapsulate ข้อมูล และโอเปอเรชันเข้าด้วยกัน (แต่ก่อนเราแยก) ในภาษาโปรแกรม เช่น C++ และ java ใช้ Class สร้าง ADT
- Monitor คือชนิดหนึ่งของ ADT ที่อนุญาตให้ เพียงเทรดเดียว Active ใน Monitor ได้ ณ เวลาใดเวลาหนึ่ง
- เทรดหลายเทรด อาจเข้าไปอยู่ใน Monitor ด้วยกันได้ แต่จะมีเพียงเทรดเดียวเท่านั้นที่ประมวลผล เทรดอื่นต้องรอ

```

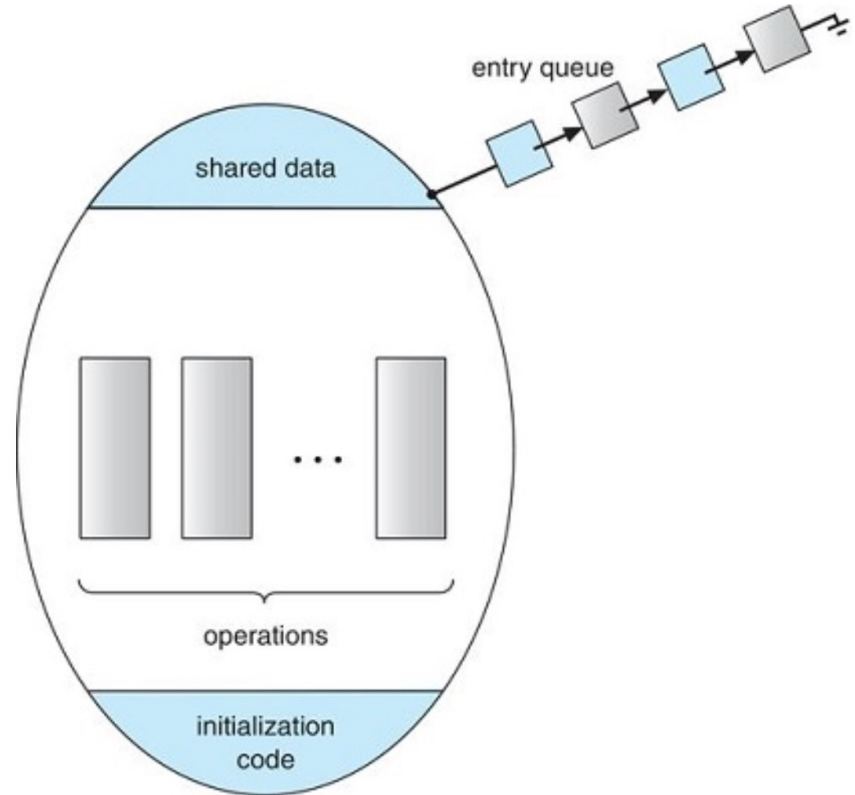
monitor monitor name
{
    /* shared variable declarations */

    function P1 ( . . . ) {
        . . .
    }

    function P2 ( . . . ) {
        . . .
    }

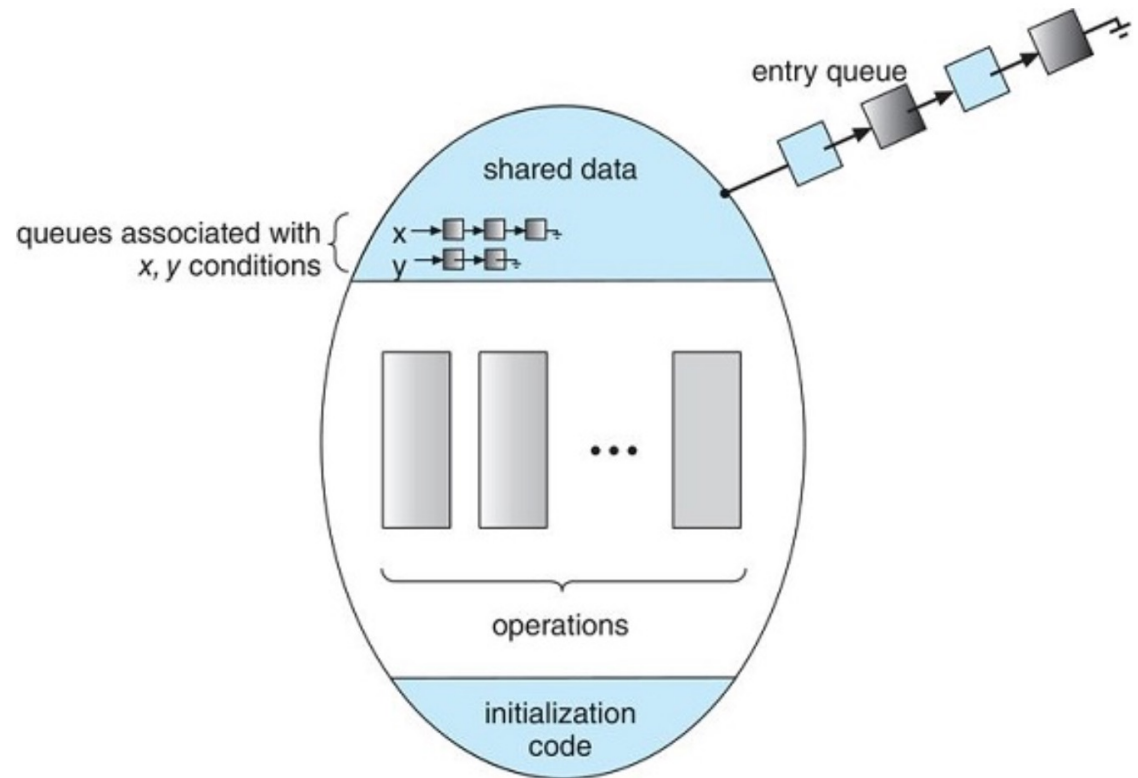
    .
    .
    .
    function Pn ( . . . ) {
        . . .
        initialization_code ( . . . ) {
            . . .
        }
    }
}

```



Monitor

- เทรดเข้าไปใน Monitor แล้วถ้าติดเงื่อนไขอะไรบางอย่างแล้วอยากจะรอจนกว่าเงื่อนไขนั้นจะเป็นจริง ก็สามารถใช้ condition variable (CV) ได้
- เป็นการทำให้ “เงื่อนไข” เป็นตัวตบขึ้นมา โดยใช้นามธรรม CV เช่น CV x
- ใช้ `x.wait()` เพื่อรอใน Q และ `x.signal()` เพื่อให้คนอื่นที่รอ x อยู่ตื่น

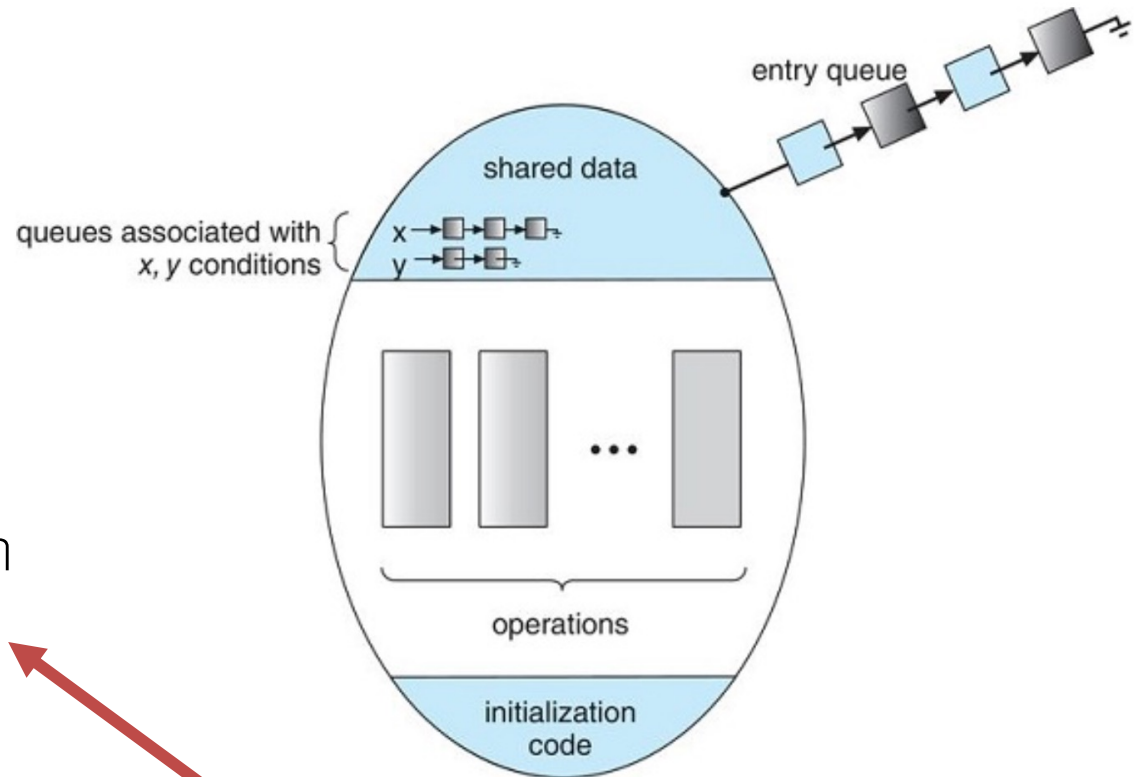


- P1 เข้าได้ คนอื่นรอข้างนอก
- P1 ทำงานต่อไม่ได้ รัน x.wait เพื่อเอาตัวเองไปรอใน

Queue x

- P2 เข้ามาได้ P2 ทำงานต่อไม่ได้ รัน x.wait เพื่อไปรอใน Queue x เหมือน P1
- P3 เข้ามาแล้ว ทำงานเสร็จ รัน x.signal ทำให้ P1 หรือ P2 อันใดอันหนึ่งตื่น สมมติว่า P2 ชนะได้รันใน Monitor
- P3 ต้องออกจาก Monitor ก่อน P2 ถึงจะรันได้

Monitor



ต้องมีกลไกป้องกันไม่ให้ สองเทรต P2 และ P3 รันใน Monitor พร้อมกัน

Mutex

- mutex คือการล็อก (lock) การเข้าใช้งาน shared resource และปลดล็อก (unlock) เมื่อใช้งาน resource นั้นเสร็จแล้ว
- ถ้ามีเธรดหนึ่งได้ lock เธรดอื่นที่จะเข้าใช้ resource ต้องรอ (block) จนกว่าเธรดที่ได้ lock จะ unlock
- ในกรณีที่มีเธรด blocked มากกว่า 1 เธรด เมื่อมีการ unlock เธรดที่ถูก block อยู่ทุกเธรดจะเปลี่ยนสถานะเป็น runnable แต่จะมีเพียงเธรดเดียวเท่านั้นที่ได้ lock และเธรดที่เหลือต้องกลับไป block รอเหมือนเดิม

ตัวแปร Mutex variable

- เป็นตัวแทนของการทำ lock/unlock synchronization
- มีชนิดเป็น `pthread_mutex_t` data type.
- เราสามารถกำหนดค่าเริ่มต้นให้เป็น `PTHREAD_MUTEX_INITIALIZER` (สำหรับตัวแปรที่ประกาศในโปรแกรม) หรือ
- กำหนดค่าเริ่มต้นด้วย `pthread_mutex_init()`
- เมื่อเลิกใช้งานให้เรียก `pthread_mutex_destroy()`

Init and destroy

```
#include <pthread.h>
```

```
int pthread_mutex_init(pthread_mutex_t *restrict mutex,  
                        const pthread_mutexattr_t *restrict attr);
```

```
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

Both return: 0 if OK, error number on failure

- Attr is set to NULL

Lock/Trylock/Unlock

```
#include <pthread.h>
```

```
int pthread_mutex_lock(pthread_mutex_t *mutex);
```

```
int pthread_mutex_trylock(pthread_mutex_t *mutex);
```

```
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

All return: 0 if OK, error number on failure

mutexex1.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <pthread.h>
5
6 pthread_mutex_t mutex_var;
7 int global_var = 100;
```

mutexex1.c

```
8
9 void * foo(void * arg){
10
11     pthread_mutex_lock(&mutex_var);
12     global_var += 25;
13     printf("foo() update global_var to %d\n", global_var);
14     pthread_mutex_unlock(&mutex_var);
15
16     pthread_exit((void *)NULL);
17 }
```

mutexex1.c

```
18
19 int main(void){
20
21     pthread_t mtid, ntid;
22     void *tret;
23
24     pthread_mutex_init(&mutex_var, NULL);
25
26     pthread_create(&ntid, NULL, foo, NULL);
27
28     pthread_mutex_lock(&mutex_var);
29     global_var += 75;
30     printf("main() update global_var to %d\n", global_var);
31     pthread_mutex_unlock(&mutex_var);
32
33     pthread_join(ntid, tret);
34
35 }
```

Mutex

	Main thread	Foo thread
	Lock(&mutex_var)	Lock(&mutex_var)
	Global_var += 75;	
	Printf("...."); // 175	
	Unlock(&mutex_var)	
		Global_var += 25;
		Printf("...."); // 200
		Unlock(&mutex_var)

Critical section

- ส่วนของโปรแกรมที่อยู่ภายใต้การ synchronization ด้วย mutex lock และ unlock operations เรียกว่า **critical section**
- มีเพียง thread เดียวเท่านั้นที่จะเข้าใช้ critical section ของตนได้

t1	t2	t3
Lock(mutexA)	Lock(mutexA)	Lock(mutexA)
Critical section ของ t1		
Unlock(mutexA)		
	Critical section ของ t2	
	Unlock(mutexA)	
		Critical section ของ t3
		Unlock(mutexA)

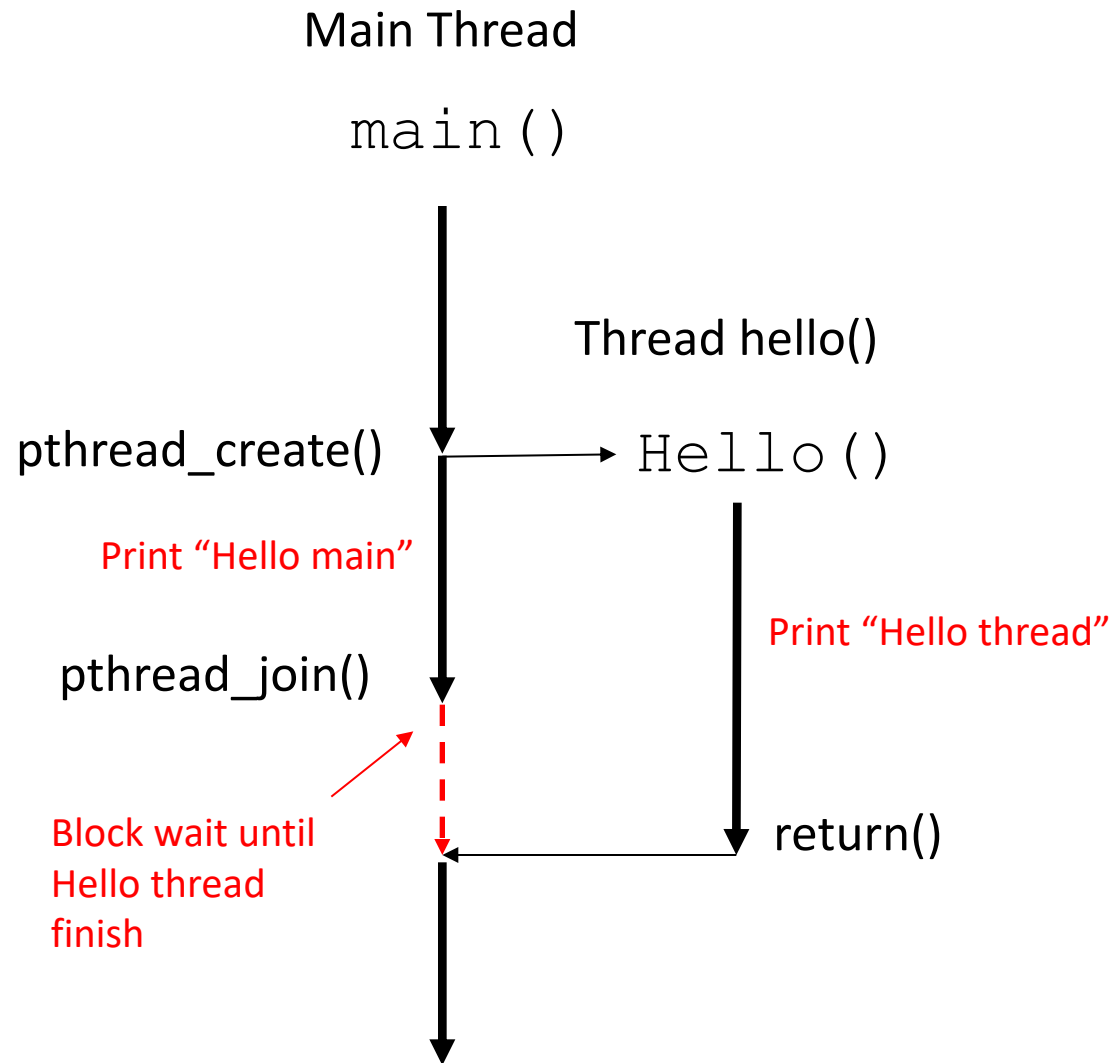
Critical section

- ส่วนใหญ่ code ใน critical section จะเป็นการเข้าถึง shared resource เช่น global variables หรือ data structures ใน heap segment e.g. `int Global_var = 0;`

t1	t2	t3
Lock(mutexA)	Lock(mutexA)	Lock(mutexA)
Global_var++		
Unlock(mutexA)		
	Global_var++	
	Unlock(mutexA)	
		Global_var++
		Unlock(mutexA)

Critical section

- Critical section เป็นปฏิปักษ์กับ parallelism
- ในการเขียนโปรแกรม multithread คุณต้องการให้หลายๆ thread ช่วยกันทำงาน
- แต่การใช้ mutex synchronization เป็นการบังคับให้ thread จำนวนหนึ่งใน process ที่ต้องการ lock ตัวแปร mutex ตัวเดียวกัน **หยุดการทำงานแบบ parallel แล้วประมวลผล critical section ของแต่ละ thread ได้ที่ละ thread เท่านั้น**



จำลอง pthread_join() ด้วย mutex

- Pthread_join() เป็นการ synchronization ระหว่าง 2 threads
- Main thread จะรอจนกระทั่ง thread ที่รอจบการทำงาน
- **ทำอะไรจึงจะทำให้ main thread รอ และ**
 - การรอด้วย mutex เกิดขึ้นเมื่อ mutex ถูก lock ไปแล้วครั้งหนึ่งและจะรอการ lock ครั้งที่สอง
- **ทำอะไรเมื่อ thread ที่ถูกรอจบการทำงานแล้วทำอะไรบางอย่างเพื่อให้ main thread หยุดรอและกลับไปทำงานคำสั่งถัดไป**
 - การหยุดรอของ thread ใดๆก็ตามที่รอ lock mutex variable หนึ่งอยู่จะเกิดขึ้น เมื่อมี thread อื่น unlock mutex variable นั้น

จำลอง join

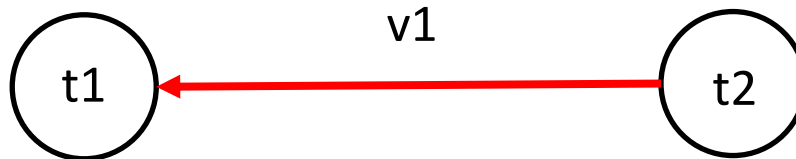
```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <pthread.h>
5
6 pthread_mutex_t mutex_var;
7
8 void * foo(void * arg){
9     pthread_t tid = pthread_self();
10    printf("foo tid = %lu do something\n", (unsigned long)tid);
11
12    sleep(5);
13
14    printf("foo unlock mutex\n");
15    pthread_mutex_unlock(&mutex_var);
16    pthread_exit((void *)NULL);
17 }
18
```

จำลอง join

```
19 int main(void){
20
21     pthread_t ntid;
22     void *tret;
23
24     pthread_mutex_init(&mutex_var, NULL);
25
26     pthread_create(&ntid, NULL, foo, NULL);
27     pthread_detach(ntid);
28     printf("main thread do something \n");
29
30     //pthread_join(ntid, tret);
31     pthread_mutex_lock(&mutex_var);
32     printf("main thread wait here...\n");
33     pthread_mutex_lock(&mutex_var);
34     printf("main thread enters critical section and leave...\n");
35     pthread_mutex_unlock(&mutex_var);
36
37 }
```

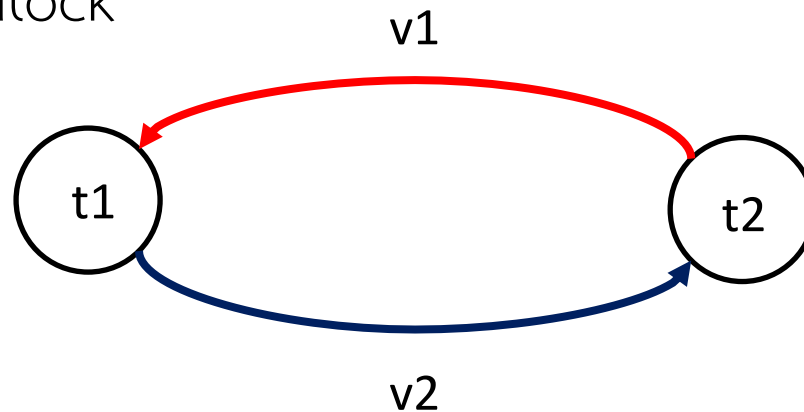
deadlock

- Deadlock คือสถานะการณ์ที่มีการ รอ กันไปมาจนหาทางออกไม่ได้
- เกิด graph แห่งการรอที่เป็น วงกลม
- กำหนดให้ เมื่อ thread t2 รอ mutex v1 ที่ thread t1 ได้ไปให้เขียนเป็น



deadlock

- Deadlock ระหว่าง thread t1 และ t2 จะเกิดขึ้นเมื่อ t2 รอ mutex v1 ที่ t1 ได้ไปและในขณะเดียวกัน t1 ก็รอ mutex v2 ที่ t2 ได้ไป ดังภาพ
- Circular wait ของการรอ อาจซับซ้อนกว่านี้เมื่อใช้ mutex หลายตัวแปร ทำให้เกิด graph แห่งการรอคอยขนาดใหญ่ แต่เมื่อใดที่มีวงกลมใน graph นั้นก็เกิด deadlock



```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <pthread.h>
5
6 pthread_mutex_t v1, v2;
7
8 void * t(void * arg){
9     long int id = (long) arg;
10    if(id == 1){
11        pthread_mutex_lock(&v1);
12        sleep(2);
13        pthread_mutex_lock(&v2);
14        printf("enter critical section of t(1)\n");
15        pthread_mutex_unlock(&v2);
16        pthread_mutex_unlock(&v1);
17    }
18    else if(id == 2){
19        pthread_mutex_lock(&v2);
20        sleep(2);
21        pthread_mutex_lock(&v1);
22        printf("enter critical section of t(2)\n");
23        pthread_mutex_unlock(&v1);
24        pthread_mutex_unlock(&v2);
25    }
26    pthread_exit((void *)NULL);
27 }
```

```
29 int main(void){
30
31     pthread_t ntid1, ntid2;
32
33     pthread_mutex_init(&v1, NULL);
34     pthread_mutex_init(&v2, NULL);
35
36     pthread_create(&ntid1, NULL, t, (void *)1);
37     pthread_create(&ntid2, NULL, t, (void *)2);
38
39     pthread_join(ntid1, NULL);
40     pthread_join(ntid2, NULL);
41
42 }
```

deadlock

- เกิด Deadlock

```
openstack@source:~/system-programming/pthread$ gcc -o e4 mutexex4-deadlock1.c -lpthread
openstack@source:~/system-programming/pthread$ ./e4
```

- ขอให้ดู code ถัดไป ถ้าเข้าไป modify ให้โปรแกรมพิมพ์ข้อมูลบนทั้งสอง
เทอร์มินัลโปรแกรมอีกครั้ง

```
9 void * t(void * arg){
10     long int id = (long) arg;
11     pthread_t tid = pthread_self();
12     printf("this is tr%ld tid = %lu \n", id, (unsigned long)tid);
13
14     if(id == 1){
15         pthread_mutex_lock(&v1);
16         pthread_mutex_lock(&v2);
17         printf("enter critical section of t(1)\n");
18         pthread_mutex_unlock(&v2);
19         pthread_mutex_unlock(&v1);
20     }
21     else if(id == 2){
22         pthread_mutex_lock(&v2);
23         pthread_mutex_lock(&v1);
24         printf("enter critical section of t(2)\n");
25         pthread_mutex_unlock(&v1);
26         pthread_mutex_unlock(&v2);
27     }
28     pthread_exit((void *)NULL);
29 }
```

deadlock

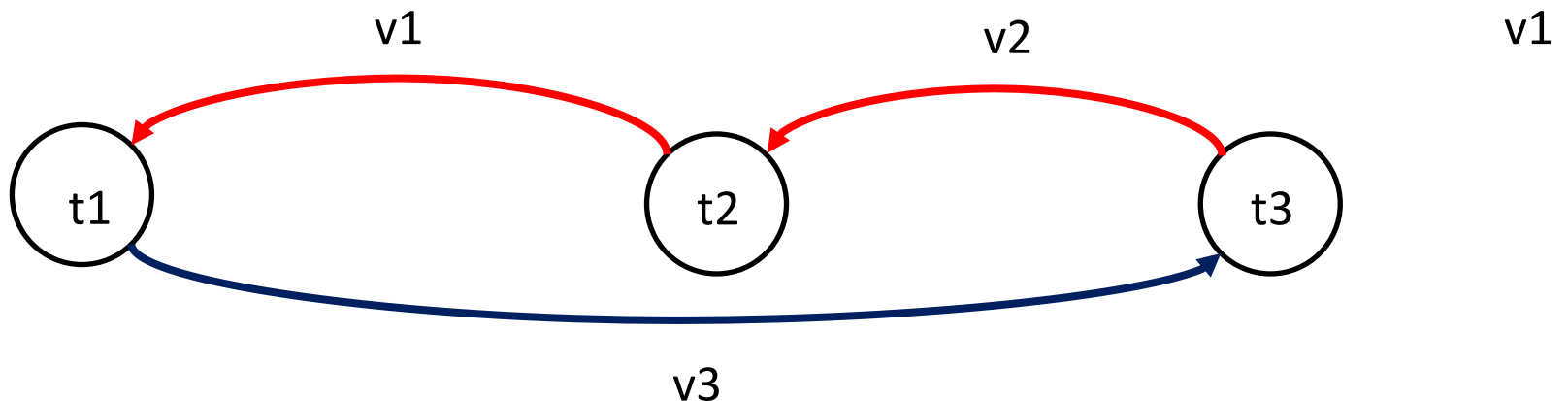
- ผลคือไม่เกิด Deadlock

```
openstack@source:~/system-programming/pthread$ gcc -o e5 mutexex5-deadlock2.c -lpthread
openstack@source:~/system-programming/pthread$ ./e5
this is tr1 tid = 140608515835648
enter critical section of t(1)
this is tr2 tid = 140608507442944
enter critical section of t(2)
openstack@source:~/system-programming/pthread$
```

- เพราะเหตุใด
- ใครเป็นผู้จัดจ้หวะการทำงานของแต่ละ thread
- ปัญหาในการเขียนโปรแกรมที่ตรวจจับได้ยากที่สุดก็คือปัญหาที่บางครั้งโปรแกรมทำงานถูกบางครั้งก็ทำงานผิดขึ้นอยู่กับสถานะแวดล้อมที่มีความไม่แน่นอน ทำให้โปรแกรมไม่มีความน่าเชื่อถือ (unreliable)
- เราต้องทำให้โปรแกรมประมวลผลอย่างเชื่อถือได้ (reliable)

Circular Waits

- Lock(v1)
- Lock(v2)
- Lock(v3)
- Lock(v3)
- Lock(v1)
- Lock(v2)



ป้องกันการเกิด deadlock (deadlock prevention)

- การแก้ปัญหา deadlock ทำได้โดยการสร้างแนวทางปฏิบัติเวลาที่โปรแกรมมีการใช้ตัวแปร mutex หลายๆ ตัวแปรในโปรแกรม
- ขอให้สังเกตว่าปัญหาเกิดขึ้นเมื่อต่างเทรดยึดตัวแปรบางตัวไว้
- t1 ยึด v1 และต้องการ v2 และ t2 ยึด v2 ต้องการ t1
- เกิดจากอันดับในการ lock ที่โปรแกรมเมอร์เขียนให้เป็นเช่นนั้น
- วิธีการแก้คือการจัดอันดับให้ไม่เกิดสถานะการัน ยึดบางตัวขึ้น
- โดยใช้ “ข้อควรปฏิบัติในการเขียน code” ที่ใช้กับทุกเทรดเหมือนกัน


```
8 void * t(void * arg){
9     long int id = (long) arg;
10    if(id == 1){
11        pthread_mutex_lock(&v1);
12        sleep(2);
13        pthread_mutex_lock(&v2);
14        printf("enter critical section of t(1)\n");
15        pthread_mutex_unlock(&v2);
16        pthread_mutex_unlock(&v1);
17    }
18    else if(id == 2){
19        pthread_mutex_lock(&v1);
20        sleep(2);
21        pthread_mutex_lock(&v2);
22        printf("enter critical section of t(2)\n");
23        pthread_mutex_unlock(&v2);
24        pthread_mutex_unlock(&v1);
25    }
26    pthread_exit((void *)NULL);
27 }
```

แก้ปัญหา deadlock

- จากภาพโปรแกรมเมอร์แก้ปัญหาโดย
- ในกรณีที่โปรแกรมมีการเรียกใช้ตัวแปร mutex หลายตัวแปรได้แก่ v1 และ v2 ถ้าเราจัดอันดับการเรียกใช้ตัวแปร mutex ของทุกเทร็ดให้เป็นในอันดับเดียวกัน ก็จะแก้ปัญหา deadlock ได้
- จากตัวอย่างก่อนหน้านี้ t(1) และ t(2) ต้องการ lock v1 ก่อนแล้วจึง lock v2 เหมือนกัน
- ดังนั้นถ้าเทร็ดใดๆในโปรแกรมต้องการ lock mutex แบบซ้อนๆกัน (nested) ก็ให้ทำตามอันดับ v1 v2 v3 ... vn ทุกเทร็ด (บางเทร็ดอาจไม่ได้ใช้หมดทุก v แต่ขอให้ทำตามอันดับ ห้ามสลับอันดับ) ก็จะปิด deadlock possibility ได้

สรุป

- แนะนำ Thread
- ชนิดของ Thread
- ทฤษฎีเกี่ยวกับ parallel processing: Amdahl's Law
- การสร้าง Thread และจบการประมวลผลของ Thread
- แนะนำ Thread Synchronization ด้วยตัวแปร Mutual Exclusion
- ศึกษาด้วยตนเอง: ปัญหา Synchronization
 - Bounded Buffer
 - Reader-Writer Problems
 - Dining Philosophers